DRESS RECOMMENDATION SYSTEM

Main Project Report

Submitted In partial fulfilment of the requirements for the degree of

BSc COMPUTER SCIENCE

of

MAHATMA GANDHI UNIVERSITY

KOTTAYAM-686560

Submitted by

SUMY JAMES (Reg No: 230021025392)

KMM COLLEGE OF ARTS AND SCIENCE

THRIKKAKARA, COCHIN-682021

MARCH–2026

KMM COLLEGE OF ARTS AND SCIENCE
Approved by AICTE & Govt. of Kerala | Affiliated to MG University, Kottayam
EDAPPALLY TOLL GATE | PIPE LINE JUNCTION | COCHIN PUBLIC SCHOOL ROAD | THRIKKAKARA | COCHIN - 682021
ACCREDITED BY NAAC WITH B GRADE
0484-2577567, 2577677, 94009 39888, 94460 97707
kmmcollege.artsandscience@gmail.com www.kmmcollege.edu.in

CERTIFICATE

This is to certify that the project report titled **"DRESS RECOMMENDATION SYSTEM"**, is the bona fide work of **SUMY JAMES (Reg. No: 230021025392)**, a student **of KMM College of Arts and Science**, undertaken in partial fulfilment of the requirement for the award of the degree **BSc Computer Science** from **Mahatma Gandhi University** during the academic year **2025-2026.**

The project work has been thoroughly reviewed and approved, as it meets the academic standards and fulfils the requisite criteria prescribed for the successful completion of the degree program.

RASMI K.P **Ms. HEERA K KARTHA**

(Project Guide) **(Head of the Department)**

Internal Examiner **External Examiner**

Place: Thrikkakara

Date:

An ISO 9001 : 2015 Certified Organiration

LCC
Computer Education

March 31, 2026

TO WHOM SO EVER IT MAY CONCERN

This is to certify that **Sumy James** 6th Semester **BSC COMPUTER SCIENCE** student **KMM College of Arts & Science, Thrikkakara** has successfully completed a project titled "**Dress Recommendation**" from our organization.

The duration of the project was for 3 months. The Project was developed in **PYTHON** and was implemented successfully.

Thanking you,

For **LCC Computer Education**

Ramaswamy
Director

LCC COMPUTER EDUCATION
LCC
COMPUTER EDUCATION

Cochin Office : 3rd Floor, Dharmodayam Building, Marine Drive, Cochin - 682 031.
Phone Off : 2525566, 2304989 E-mail: info@lcccochin.com

DECLARATION

I hereby declare that the project report entitled **"DRESS RECOMMENDATION SYSTEM"** has been completed under the guidance of **MRs. RASMI K.P**, and submitted to the Department of Computer Science, **KMM College of Arts and Science**, **Thrikkakara**, in partial fulfilment of the requirements for the award of the degree of **BSc Computer Science.** This report is an authentic record of my original work and has not been submitted for the award of any degree at this university or any other university.

Date:

Place: Thrikkakara **SUMY JAMES(Regno: 230021025392)**

ACKNOWLEDGEMENT

It is a great privilege to present my project, **“DRESS RECOMMENDATION SYSTEM".** First and foremost, I express my sincere gratitude to **God Almighty** for his countless blessings, guidance, and strength throughout this endeavour.

I take this opportunity to extend my heartfelt thanks to **Dr. Sabana Backer, Principal of KMM College of Arts and Science**, for her unwavering support and encouragement.

My deepest appreciation goes to **Ms. Heera K Kartha, Head of the Computer Science Department**, for her invaluable guidance and motivation.

I am profoundly grateful to **MRs. RASMI K P**, **my project guide**, for her continuous support, insightful advice, and valuable suggestions, which played a pivotal role in the successful completion of this project.

Furthermore, I extend my sincere thanks to all the **teaching and non-teaching staff** who have directly or indirectly assisted me throughout this journey.

A special note of appreciation goes to my **parents**, whose moral support and encouragement have been my greatest source of strength. Lastly, I extend my gratitude to my **friends**, with whom I shared my experiences and received valuable suggestions that significantly enhanced the quality of my work.

SUMY JAMES

SYNOPSIS

The Dress Recommendation System is an innovative web-based platform designed to provide personalized outfit suggestions by analyzing user characteristics such as body type, skin tone, preferences, and occasion. In the modern digital era, fashion choices have become an important part of personal identity, yet many users struggle to find dresses that truly suit them. Online shopping platforms offer a large number of dress collections, but they rarely consider individual differences, making it difficult for users to make confident decisions. This system addresses this gap by introducing an intelligent recommendation approach that focuses on personalization and user satisfaction.

The system works by collecting user inputs such as measurements, color preferences, style choices, and the purpose of the outfit. Based on this information, the platform suggests dresses that match the user's profile, reducing the time and effort required to search for suitable outfits. Users can also explore various dress templates and customize them according to their taste, which makes the platform more interactive and engaging. This personalized approach helps users feel more confident in their fashion choices and improves their overall online shopping experience.

In addition to user features, the system provides dedicated dashboards for designers and administrators to ensure smooth system management. Designers can upload new dress templates, update existing designs, and manage collections, while the admin monitors user activities and maintains the database. The inclusion of rating features and template management enhances the reliability and usability of the system. Overall, the Dress Recommendation System combines personalization, customization, and efficient management to create a modern solution that simplifies dress selection and supports better fashion decisions.

TABLE OF CONTENTS

NO	CONTENTS	PAGE
1	**INTRODUCTION**	1
	1.1 OVERVIEW OF THE PROJECT.	2
	1.2 PROBLEM DEFINITION AND OBJECTIVE OF THE PROJECT	2
2	**SYSTEM ANALYSIS**	4
	2.1 INTRODUCTION	5
	2.2 IDENTIFICATION OF THE NEED	6
	2.3 EXISTING SYSTEM AND ITS LIMITATIONS	6
	2.4 PROPOSED SYSTEM	8
	2.4.1 BENEFITS OF PROPOSED SYSTEM	10
	2.5 FEASIBILITY ANALYSIS	10
	2.5.1 ECONOMIC FEASIBILITY	11
	2.5.2 TECHNICAL FEASIBILITY	11
	2.5.3 OPERATIONAL FEASIBILITY	11
	2.6 SYSTEM SPECIFICATION	12
	2.6.1 SOFTWARE SPECIFICATION	19
	2.6.2 HARDWARE SPECIFICATION	20

	2.6.3 TOOLS/PLATFORM	20
3	**SYSTEM DESIGN**	21
	3.1 INTRODUCTION	22
	3.2 INPUT DESIGN	23
	3.3 OUTPUT DESIGN	23
	3.4 DATABASE DESIGN	24
	3.5 DATA FLOW DIAGRAM	26
	3.6 E-R DIAGRAM	32
	3.7 USE CASE DIAGRAM	35
4	**SYSTEM DEVELOPMENT**	39
	4.1 MODULE DESCRIPTION	40
5	**SYSTEM IMPLEMENTATION**	42
	5.1 TESTING	43
	5.2 VALIDATION CHECK	45
	5.2 TEST CASE	46
	5.3 SYSTEM IMPLEMENTATION	47
	5.4 SECURITY	47

6	**SYSTEM MAINTENANCE AND FUTURE ENHANCEMENT**	49
	6.1 SYSTEM MAINTENANCE	50
	6.2 FUTURE ENHANCEMENTS	52

7	**CONCLUSION**	54
8	**APPENDIX**	56
	8.1. APPENDIX A. TABLE DESIGN	57
	8.2 APPENDIX B. SAMPLE INPUT AND OUTPUT DESIGN	61
	8.3 APPENDIX C. SOURCE CODE	69
9	**BIBLIOGRAPHY**	87

1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

The Dress Recommendation System is a web-based application developed to provide smart and personalized dress suggestions to users based on their individual characteristics and preferences. The system is designed to reduce the difficulty users face while selecting outfits from a large collection of dresses available online. By collecting details such as body measurements, skin tone, preferred colors, and occasion, the system analyzes user input and provides suitable dress recommendations that match their style and requirements.

The project consists of three main modules: User Module, Designer Module, and Admin Module. The user module allows individuals to register, log in, enter personal details, browse dress templates, customize outfits, and rate designs. The designer module enables designers to upload new dress templates, manage collections, and update fashion designs regularly. The admin module monitors the entire system, manages users and designers, and ensures smooth functioning of the platform. Through these modules, the system creates a complete environment for personalized dress recommendation and efficient system management.

1.2 PROBLEM DEFINITION AND OBJECTIVE OF THE PROJECT

Problem Definition: In the present online fashion industry, users are provided with a large number of dress options on shopping platforms, but they often struggle to find outfits that truly suit their body type, skin tone, and personal preferences. Most existing platforms display general product listings without considering individual differences, which makes the selection process confusing and time-consuming. As a result, users spend a lot of time browsing through multiple products and still remain unsure about their final choice.

Another major problem is the lack of customization and interaction between users and designers. Users cannot easily modify dress designs according to their needs, and there is no dedicated system to provide personalized fashion suggestions. This results in poor user satisfaction and an inefficient shopping experience. Therefore, there is a need for a smart system that can provide personalized dress recommendations and allow users to customize designs based on their preferences.

Objective: The primary objective of the Dress Recommendation System is to develop an intelligent platform that provides personalized dress suggestions based on user details such as body measurements, skin tone, style preferences, and occasion. The system aims to reduce the time and

effort required for users to select suitable outfits and improve their confidence in choosing dresses online.

Another objective of the project is to create an interactive platform that supports customization and efficient system management. The system allows users to modify dress templates, rate designs, and explore new fashion trends. It also provides separate dashboards for designers and administrators to manage templates, users, and system activities. By achieving these objectives, the project aims to enhance user satisfaction and make the dress selection process simple, fast, and personalized.

2. SYSTEM ANALYSIS

2.1 INTRODUCTION

The fashion industry has experienced significant growth with the rise of e-commerce platforms and digital technologies. Today, many people prefer online shopping because it offers convenience, variety, and easy access to the latest trends. Despite these advantages, choosing the right dress remains a difficult task for many users. Online platforms typically display a large number of dress options without considering individual characteristics such as body type, skin tone, personal style, and occasion. As a result, users often feel overwhelmed by the choices available and may end up selecting outfits that do not suit them well.

In traditional shopping, customers could try dresses before purchasing, but this option is limited in online shopping environments. Without proper guidance, users rely only on product images and descriptions, which may not accurately represent how the dress will look on them. This leads to uncertainty, dissatisfaction, and sometimes incorrect purchase decisions. Therefore, there is a need for a system that can guide users and help them make better fashion choices based on their personal attributes and preferences.

The Dress Recommendation System is designed to overcome these challenges by introducing personalization into the online dress selection process. The system collects essential user information such as body measurements, preferred colors, and the purpose of the outfit, and then analyzes this data to generate suitable dress recommendations. By using intelligent recommendation techniques, the system reduces the time and effort required to search for the perfect outfit and increases user confidence in their selection.

In addition to providing recommendations, the system also allows users to customize dress templates according to their taste. This feature makes the platform more interactive and user-friendly, as users can modify designs and explore different styles before making a decision. The system also includes separate dashboards for designers and administrators, enabling them to manage dress collections, monitor system activities, and maintain the platform efficiently. Overall, the Dress Recommendation System aims to enhance the online shopping experience by making dress selection simple, personalized, and convenient for users.

2.2 IDENTIFICATION OF NEED

In today's fast-paced lifestyle, people expect quick and accurate solutions that save time and effort. When it comes to fashion shopping, users want to find dresses that suit them perfectly without spending hours searching through multiple websites. However, the absence of personalized guidance in most online platforms makes the process complicated and sometimes frustrating. Users often depend on guesswork while selecting dresses, which may lead to poor purchasing decisions and dissatisfaction with the final outcome.

Another important need arises from the lack of interaction between users and designers. Many users have unique preferences and ideas but do not have the opportunity to communicate these requirements directly to designers. Without customization features, users are limited to choosing only the available designs. This highlights the importance of providing a system that bridges the gap between users and designers by offering customization and template-based modifications.

Furthermore, the growth of digital technologies and artificial intelligence has created opportunities to develop intelligent recommendation systems. Implementing such technology in the fashion industry can significantly improve the way users discover and select outfits. A smart recommendation platform can analyze user data and provide accurate suggestions, making the entire process more efficient and reliable.

Therefore, the need for the Dress Recommendation System becomes very clear. The system aims to provide personalized recommendations, customization options, and efficient management through dedicated dashboards for users, designers, and administrators. By addressing these needs, the system contributes to a more satisfying and convenient online fashion shopping experience.

.

2.3 EXISTING SYSTEM AND IT'S LIMITATIONS

In the current fashion e-commerce environment, users rely mainly on general online shopping platforms to browse and purchase dresses. These platforms display a large collection of outfits categorized by brand, price, or trend, but they do not focus on individual user characteristics. Users must manually search through hundreds of products and decide which dress might suit them based on images and descriptions. This process is time-consuming and often confusing, especially for users who are unsure about their body type, skin tone, or personal style.One of the major limitations of the

existing system is the absence of personalized recommendations. Most platforms provide suggestions based on popularity or previous purchases rather than analyzing personal attributes such as body measurements or occasion. As a result, users may receive irrelevant suggestions that do not match their needs. Additionally, there is very limited support for dress customization, which prevents users from modifying designs according to their preferences.Another drawback is the lack of communication between users and designers. Users cannot directly interact with designers or request changes to dress templates.

The current online fashion shopping platforms provide a wide range of dress collections, but they have several limitations that affect user satisfaction and decision-making. The major limitations of the existing system are listed below:

1. **Lack of Personalization**

- Most online platforms provide general recommendations based on popularity or sales trends rather than individual user characteristics. They do not consider important factors such as body type, skin tone, and personal preferences while suggesting dresses.

2. **Time-Consuming Search Process**

- Users need to browse through a large number of dress options manually. This process takes a lot of time and effort, and users often feel confused due to the overwhelming number of choices available.

3. **No Dress Customization Feature**

- Existing systems do not provide proper customization options. Users cannot easily modify dress designs according to their needs, which limits their ability to get outfits that match their personal style.

4. **Lack of Interaction with Designers**

- There is no direct communication between users and designers. Users cannot request changes or provide feedback on dress designs, which reduces the flexibility of the shopping experience.

5. Irrelevant Recommendations

- Recommendations are usually based on general browsing history instead of detailed user input. This results in suggestions that may not suit the user's appearance or occasion.

6. No Dedicated Management System

- There is no proper system to manage users, designers, templates, and feedback in a centralized manner. This makes it difficult to maintain and update the platform efficiently.

7. Reduced User Satisfaction

- Due to the lack of personalization and customization, users may feel dissatisfied with their choices and may lose confidence in online fashion shopping platforms.

These limitations clearly indicate the need for a smart Dress Recommendation System that provides personalized suggestions, customization features, and efficient system management.

2.4 PROPOSED SYSTEM

The proposed Dress Recommendation System is a smart and user-friendly web application designed to provide personalized dress suggestions based on individual user characteristics and preferences. The system aims to overcome the limitations of existing online shopping platforms by introducing intelligent recommendation techniques and customization features. By collecting user inputs such as body measurements, skin tone, preferred colors, and occasion, the system generates suitable dress recommendations that match the user's needs and improve their shopping experience.

The system includes three main modules: User Module, Designer Module, and Admin Module. The User Module allows users to register, log in, enter personal details, browse dress templates, customize outfits, and rate designs. The Designer Module enables designers to upload new dress templates, update existing designs, and manage dress collections efficiently. The Admin Module monitors the entire system, manages users and designers, and maintains the database. Through these modules, the system ensures smooth operation and effective management.

.

Key Components of the Proposed System

The Dress Recommendation System is designed with several important components that work together to provide personalized recommendations and ensure smooth system operation. Each component plays a specific role in handling user interaction, data processing, and system management.

1. **User Interface**

- The user interface provides a simple and interactive platform where users can register, log in, enter personal details, browse dress templates, and receive recommendations. It ensures easy navigation and improves the overall user experience.

2. **User Profile Management**

- This component stores and manages user information such as body measurements, skin tone, style preferences, and occasion details. The collected data is used by the system to generate accurate dress recommendations.

3. **Recommendation Engine**

- The recommendation engine is the core component of the system. It analyzes user inputs and generates personalized dress suggestions based on preferences, body type, and occasion. This component helps reduce the time required to search for suitable outfits.

4. **Dress Customization Module**

- This module allows users to modify dress templates according to their preferences. Users can explore different styles and make changes to designs, making the platform more interactive and flexible.

5. **Designer Module**

- The designer module enables designers to upload new dress templates, update existing designs, and manage dress collections. This ensures that the system always has updated and trendy fashion options.

6. Admin Module

- The admin module is responsible for monitoring and managing the entire system. It helps in managing users, designers, templates, and feedback to ensure smooth functioning of the platform.

7. Database Management System

- The database stores user details, dress templates, ratings, and system data securely. It ensures quick data retrieval and efficient system performance.

These components work together to create a reliable and efficient Dress Recommendation System that provides personalized and customizable dress suggestions.

2.4.1 BENEFITS OF PROPOSED SYSTEM

The proposed Dress Recommendation System provides several advantages that improve the overall online dress selection experience. By introducing personalization and customization features, the system helps users make better fashion decisions quickly and easily.

2.5 FEASIBILITY STUDY

During the system analysis, a feasibility study of the proposed Dress Recommendation System is carried out to determine whether the system is beneficial and practical for development. It is necessary to evaluate the feasibility of the project at an early stage with minimum cost and effort. The feasibility study acts as a test for system acceptance, its workability, its ability to meet user requirements, and the effective use of available resources. The main steps involved in the feasibility analysis are:

- Formation of a project team
- Preparing the system flow and basic architecture
- Identifying possible system solutions
- Selecting the most suitable solution for development

The proposed system helps users find suitable dresses more efficiently and accurately by providing personalized recommendations and customization features. The results obtained after conducting the feasibility study are as follows:

- Economic Feasibility
- Technical Feasibility
- Operational Feasibility

2.5.1 ECONOMIC FEASIBILITY

Economic feasibility evaluates whether the project can be developed within a reasonable budget and whether the benefits outweigh the costs. The Dress Recommendation System uses open-source technologies and free development tools, which significantly reduces the cost of development. There is no need for expensive hardware or licensed software, making the project financially manageable.

2.5.2 TECHNICAL FEASIBILITY

Technical feasibility focuses on whether the required technology, tools, and resources are available to develop the system. The Dress Recommendation System can be implemented using widely used web technologies such as front-end frameworks, back-end programming languages, and database management systems. These technologies are well supported and have a large developer community, making the development process easier and more reliable. The system does not require specialized hardware, and it can operate efficiently on standard computers and web servers.

2.5.3 OPERATIONAL FEASIBILITY

Operational feasibility determines whether the system will be accepted and used effectively by its target users. The Dress Recommendation System is designed with a simple and user-friendly interface, making it easy for users, designers, and administrators to interact with the platform. Users can easily register, enter their details, and receive dress recommendations without needing advanced technical knowledge.

The system also provides separate dashboards for designers and administrators, allowing them to manage templates, users, and system activities efficiently. Proper data storage and management ensure smooth system performance and reliability. Since the system is easy to use and meets the needs of its users, it is considered operationally feasible and suitable for real-world implementation.

2.6 SYSTEM SPECIFICATION

PYTHON

Python is a general-purpose interpreted, interactive, object-oriented, and high- level programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). This tutorial gives enough understanding on Python programming language. Python is a high-level, interpreted, interactive and object- oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

Python is a must for students and working professionals to become a great Software Engineer especially when they are working in Web Development Domain. I will list down some of the key advantages of learning Python. Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands. Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, Smalltalk, and UNIX shell and other scripting languages. Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL). Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python is often described as a "batteries included" language due to its comprehensive standard library. Python was conceived in the late 1980s as a successor to the ABC language. Python 2.0, released in 2000, introduced features like list comprehensions and a garbage collection system capable of collecting reference cycles. Python 3.0, released in 2008, was a major revision of the language that is not completely backward-compatible, and much Python 2 code does not run unmodified on Python 3. The Python 2 language, i.e. Python 2.7.x, was officially discontinued on 1 January 2020 (first planned for 2015) after which security patches and other improvements will not be released for it. With Python 2's end- of-life, only Python 3.5.x and later are supported. Python interpreters are available for

many operating systems. A global community of programmers develops and maintains Python, an open-source reference implementation. A non-profit organization, the Python Software Foundation, manages and directs resources for Python and Python development.

Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python is often described as a "batteries included" language due to its comprehensive standard library. Python was conceived in the late 1980s as a successor to the ABC language. Python 2.0, released in 2000, introduced features like list comprehensions and a garbage collection system capable of collecting reference cycles. Python 3.0, released in 2008, was a major revision of the language that is not completely backward-compatible, and much Python 2 code does not run unmodified on Python 3. The Python 2 language, i.e. Python 2.7.x, was officially discontinued on 1 January 2020 (first planned for 2015) after which security patches and other improvements will not be released for it. With Python 2's end- of-life, only Python 3.5.x and later are supported. Python interpreters are available for many operating systems. A global community of programmers develops and maintains Python, an open-source reference implementation. A non-profit organization, the Python Software Foundation, manages and directs resources for Python and Python development.

Python strives for a simpler, less-cluttered syntax and grammar while giving developers a choice in their coding methodology. In contrast to Perl's "there is more than one way to do it" motto, Python embraces a "there should be one and preferably only one obvious way to do it" design philosophy. Alex Martello, a Fellow at the Python Software foundation and Python book author, writer that “To describe something as ‘clever’ is not considered a compliment in the python culture.” Python’s developers strive to avoid premature optimization, and reject patches to non-critical part of the Python reference implementation that would offer marginal increases in speed at the cost of clarity.

Flask

Flask is a lightweight, open-source web framework written in Python. It is designed to make getting started with web development quick and easy, while also being highly flexible and scalable for more complex applications. Flask was created by Armin Ronacher of Pocoo, an international

group of Python enthusiasts formed in 2004. According to Ronacher, the idea was originally an April Fool's joke that was popular enough to make into a serious application. The name is a play on the earlier Bottle framework.

Flask follows the WSGI (Web Server Gateway Interface) standard and the Model-View-Controller (MVC) architecture. It is classified as a micro-framework because it does not include built-in tools or libraries like form validation, database abstraction layer, or authentication by default. However, Flask is highly extensible, allowing developers to integrate only the components they need through third party libraries and plugins. Flask is designed to be simple and unopinionated, giving developers the freedom to choose their tools and libraries.

Key Features of Flask

1. **Modular and Extensible**

- Supports easy integration with third-party libraries and extensions such as Flask-SQL Alchemy, Flask-Login, Flask-WTF, etc.

2. **Jinja2 Templating Engine**

- Provides a powerful templating system to dynamically generate HTML pages with embedded Python logic.

3. **RESTful Request Handling**

- Supports HTTP methods like GET, POST, PUT, DELETE—ideal for building REST APIs.

4. **Cross-Platform and Scalable**

- Flask apps can run on Windows, macOS, Linux, or cloud platforms like Heroku, AWS, GCP, etc.

5. **Flexible with Backend and Frontend**

- Compatible with any frontend framework (React, Vue, Bootstrap) and backend technologies (SQL or NoSQL databases).

Hypertext Mark-up Language (HTML)

It is the standard mark-up language for documents designed to be displayed in a web browser. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.
Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.

HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page. HTML provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes and other items. HTML elements are delineated by tags, written using angle brackets.

Tags such as <imp /> and <input /> directly introduce content into the page. Other tags such as <p> surround and provide information about document text and may include other tags as sub-elements. Browsers do not display the HTML tags but use them to interpret the content of the page. HTML can embed programs written in a scripting language such as JavaScript, which affects the behaviour and content of web pages. Inclusion of CSS defines the look and layout of content.

JavaScript

It is a lightweight, interpreted, object-oriented language with first-class functions, and is best known as the scripting language for Web pages, but it's used in many non-browser environments as well. It is a prototype-based, multi- paradigm scripting language that is dynamic, and supports object oriented, imperative, and functional programming styles. JavaScript runs on the client side of the web, which can be used to design / program how the web pages behave on the occurrence of an event. JavaScript is an easy to learn and powerful scripting language, widely used for controlling web page behaviour.

JavaScript can function as both a procedural and an object-oriented language. Objects are created programmatically in JavaScript, by attaching methods and Properties to otherwise empty objects at run time, as opposed to the syntactic class definitions common in compiled languages like C++ and Java. Once an object has been constructed it can be used as a blueprint (or prototype) for creating similar objects.

Cascading Style Sheets (CSS)

It is a style sheet language used to describe the presentation of a document written in HTML or XML (including XML dialects such as SVG, Math or XHTML). CSS describes how elements should be rendered on screen, on paper, in speech, or on other media.

MYSQL

MySQL is an Oracle-backed open-source relational database management system (RDBMS) based on Structured Query Language (SQL). MySQL runs on virtually all platforms, including Linux, UNIX and Windows. Although it can be used in a wide range of applications, MySQL is most often associated with web applications and online publishing.

MySQL is an important component of an open-source enterprise stack called WAMP. WAMP is a web development platform that uses Linux as the operating system, Apache as the web server, and MySQL as the relational database management system and PHP as the object-oriented scripting language. (Sometimes Perl or Python is used instead of PHP.)

Originally conceived by the Swedish company MySQL AB, MySQL was acquired by Sun Microsystems in 2008 and then by Oracle when it bought Sun in 2010. Developers can use MySQL under the GNU General Public License (GPL), but enterprises must obtain a commercial license from Oracle. Today, MySQL is the RDBMS behind many of the top websites in the world and countless corporate and consumer-facing web-based applications, including Facebook, Twitter and YouTube.
MySQL is the database management system, or a database server.

How MySQL works

MySQL is based on a client-server model. The core of MySQL is MySQL server, which handles all of the database instructions (or commands). MySQL server is available as a separate program for use in a client-server networked environment and as a library that can be embedded (or linked) into separate applications.
MySQL operates along with several utility programs which support the administration of MySQL databases. Commands are sent to MySQL Server via the MySQL client, which is installed on a computer.

MySQL was originally developed to handle large databases quickly. Although MySQL is typically installed on only one machine, it is able to send the database to multiple locations, as users are able to access it via different MySQL client interfaces. These interfaces send SQL statements to the server and then display the results.

MySQL Features

Relational Database Management System (RDBMS): MySQL is a relational database management system

- Easy to use: MySQL is easy to use. You have to get only the basic knowledge of SQL. You can build and interact with MySQL with only a few simple SQL statements.
- It is secure: MySQL consist of a solid data security layer that protects sensitive data from intruders. Passwords are encrypted in MySQL.
- Client/ Server Architecture: MySQL follows a client /server architecture. There is a database server (MySQL) and arbitrarily many clients (application programs), which communicate with the server; that is, they query data, save changes, etc.
- Free to download: MySQL is free to use and you can download it from MySQL official website.
- It is scalable: MySQL can handle almost any amount of data, up to as much as 50 million rows or more. The default file size limit is about 4 GB. However, you can increase this number to a theoretical limit of 8 TB of data.
- Compatible on many operating systems: MySQL is compatible to run on many operating systems, like Novell NetWare, Windows* Linux*, many varieties of UNIX* (such as Sun* Solaris*, AIX,

and DEC* UNIX), OS/2, FreeBSD*, and others. MySQL also provides a facility that the clients can run on the same computer as the server or on another computer (communication via a local network or the Internet).

- Allows roll-back: MySQL allows transactions to be rolled back, commit and crash recovery.
- High Performance: MySQL is faster, more reliable and cheaper because of its unique storage engine architecture.
- High Flexibility: MySQL supports a large number of embedded applications which makes MySQL very flexible.
- High Productivity: MySQL uses Triggers, Stored procedures and views which allows the developer to give a higher productivity.

Core MySQL features

MySQL enables data to be stored and accessed across multiple storage engines, including Inorb, CSV, and NDB. MySQL is also capable of replicating data and partitioning tables for better performance and durability. MySQL users aren't required to learn new commands; they can access their data using standard SQL commands.

Before 2016, the main difference between MySQL and SQL was that the former could be used on multiple platforms, whereas the latter could only be used on Windows. Microsoft has since expanded SQL to support Linux, a change which went into effect in 2017. When MySQL is installed via Linux, its package management system requires custom configuration to adjust security and optimization settings.

MySQL also allows users to choose the most effective storage engine for any given table, as the program can utilize multiple storage engines for individual tables. One of MySQL's engines is Inorb. Inorb was designed for high availability. Because of this, it is not as quick as other engines. SQL uses its own storage system, but it does maintain multiple safeguards against loss of data. Both systems are able to run in clusters for high availability.
SQL Server offers a wide variety of data analysis and reporting tools. SQL Server Reporting Services is the most popular one and is available as a free download. There are similar analysis

tools for MySQL available from third-party software companies, such as Crystal Reports XI and Actuate BIRT.

What is Windows?

Windows 10 professional integrates the strengths of windows 2008 professional such as standards-based security, manageability, and reliability, with the best business features of windows 98 and windows Millennium Edition, such as plug and play, simplified user interface, and innovative support services. This combination creates the best desktop operating system for business.
It is more users friendly and a stable operating system equipped with much more added features. The operating system supports new technologies such as digital video disks, multiple monitors etc. along with plug and play and multi display features. It has a graphical user interface operating environment. Faster computing, easy access to remote information and control remote computers are some added features. Following are the common features of Windows 10.

Faster computing, easy access to remote information and control remote computers. Built in networking and messaging facility.

- Easier to set up, add or remove.
- Increase system security and control.
- Support advanced networking and communication.

2.6.1 SOFTWARE SPECIFICATIONS

The software requirements specification (SRS) is a means of translating the ideas in the minds of clients into a formal documentation. This document forms the development and software validation. The basic reason for the difficulty in software requirement specification comes from the fact that there are three interested parties-the clients, the end users and the software developer. The requirements document has to be such that the client and the user can understand easily, and the developers can use it as a basis for software development. Due to the diverse parties involved in software requirement specification, a communication gap exists. This gap arises when the client does not understand software or the software development processor when the developer does not understand the client's problem and application area of SRS bridges this communication gap.

Problem analysis is done to obtain a clear understanding of the needs of the clients and the users, and what exactly is desired from the software. Analysis leads to the actual specification. People performing the analysis called analysts, area also responsible for specifying the requirements.

The software project is initiated by the client's needs. In the beginning these needs are in the minds of various people in the client organization. The requirement analyst must identify their requirements by talking to these people and understanding their needs. These people and the existing documents about the current mode of operation are the basis source of information for the analyst.

2.6.2 HARDWARE SPECIFICATIONS

Processor	Pentium 4 Processor or Above
Memory	4 GB or Above
Hard Disk	40 GB or Above
Keyboard	Any Compatible
Mouse	Any Compatible

2.6.3 TOOL SPECIFICATIONS

Operating System	Windows 8 or Above
Web Technologies	Python, Flask, HTML, CSS
Database	MySQL
Web Browser	Google Chrome/Mozilla Firefox

3. SYSTEM DESIGN

3.1 INTRODUCTION

System design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. System design could be seen as the application of systems theory to product development. There is some overlap with the disciplines of system analysis, systems architecture and systems engineering.

TYPES OF SYSTEM DESIGN

Logical Design

Logical design pertains to an abstract representation of the data flow, inputs and outputs of the system. It describes the inputs (source), outputs (destinations), databases (data stores), procedures (data flows) all in a format that meets the user requirements.

Physical

Physical design relates to the actual input and outputs processes of the system. It focuses on how data is entered into a system, verified, processed, and displayed as output.

Architectural Design

It is also known as high level design that focuses on the design of system architecture. It describes the structure and behaviour of the system. It defines the structure and relationship between various modules of system development process.

Detailed Design

It follows Architectural design and focuses on development of each module.

Conceptual Data Modelling

It is representation of organization data which includes all the major entities and relationship. System analyses develop a conceptual data model for the current system that supports the scope and requirement for the proposed system.

3.2 INPUT DESIGN

Input design is a part of the overall design. The input methods can be broadly classified. Internal controls must be established for monitoring the number of inputs and for ensuring that the data are valid. The basic steps involved in input design are:

- Review input requirements.
- Decide how the input data flow will be implemented.
- Decide the source document.
- Prototype online input screens.
- Design the input screens.

The quality of the system input determines the quality of the system output. Input specifications describe the way data enter the system for processing. Input design features can ensure the reliability of the system and produce result.

3.3 OUTPUT DESIGN

A quality is one, which meets the requirements of end user and present the information clearly. In any system results of processing are communicated to the user and to the other system through outputs. In the outputs design it is determined how the information is to be displayed for immediate need.

It is the most important and direct source information is to user. Efficient and intelligent output design improves the system's relationships with the user and helps in decision making. The objectives of the output design are to convey the information of all the past activities, current status and to emphasis important events. The output generally refers to the results and information that is generated from the system. Outputs from computes are required primarily to communicate the result of processing to the users.

Output also provides a means of storage by coping the results for later reference in consolation. There is a chance that some of the end users will not actually operate the input data or information through workstations but will see the output from the system.

Two phases of the output design are:

1. Output Definitions
2. Output Specification

Output definitions consider the type of outputs contents, its frequency and its volume, the appropriate outputs media is determined for output. Once the media is chosen, the details specification of output documents are carried out. The nature of output required from the proposed system is determined during logical design stage. It takes the outline of the output from the logical design and produces output as specified during the logical design phase. In a project, when designing the output, the system must accomplish the following:

- Determine the information to present.
- Decide whether to display, speak the information and select the output medium.
- Arrange the information in acceptable format.
- Decide how to distribute the output to the intended receipt.

Thus, by following the above specification, a high-quality output can be generated. Outputs from compute system are required primarily to communicate thee result of processing to users. Computer output is the most important and direct source of information to the user. Efficiency, intelligible output should improve the system's relationship with the user and help in decision making. The output devices to consider depend on factors as compatibility of the device with the system, response time requirements, expected print quality, number of copies needed etc.

3.4 DATABASE DESIGN

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make information access easy, quick, inexpensive and flexible for users. The general theme behind a database is to integrate all information. Database design is recognized as a standard of management information system and is available virtually for every computer system. In database design several specific objectives are considered:

- Ease of learning and use
- Controlled redundancy
- Data independence
- More information at low cost
- Accuracy and integrity
- Recovery from failure
- Privacy and security

NORMALIZATION

Designing a database is complete task and the normalization theory is a useful aid in the design process. The process of normalization is concerned with transformation of conceptual schema into computer representation form. There will be need for most databases to grow by adding new attributes and new relations. The data will be used new ways. Tuple will be added and deleted. Information stored may undergo updating also. New association may also be added. In such situation the performance of a database is entirely depend upon its design. A bad database design may lead to certain undesirable things like:

- Repetition of information
- Inability to represent certain information.
- Loss of information

To minimize these anomalies, normalization may be used. If the database is in a normalized from, the data can be growing without, in most cases, forcing the rewriting application programs. This is important because of the excessive and growing cost of maintaining an organization's application programs and its data from the disrupting effects of database growth. As the quality of application programs increase, the cost of maintaining the without normalization will rise to prohibitive levels, A normalized database can also encompass many related activities of an organization thereby minimizing the need for rewriting the applications of programs. Thus, normalization helps one attain a good database design and there by ensures continued efficiency of database.

We can define the procedure as the successive reduction of a given collection of relations to some more desirable from. This procedure is reversible. That is, it is always possible to take the output from the procedure and convert them back into input. In this process, no information is lost. So, it is also called “no lose decomposition”.

FIRST NORMAL FORM: A relation is first normal form (1NF) if and all its attributes are based on single domain. The objective of normalization a table is to remove its repeating groups and ensure that all entries of the resulting table have at most single value.

SECOND NORMAL FORM: A table is said to be second normal form (2NF), when it is in 1NF and every attribute in the record is functionally dependent upon the whole key, and not just a part of the key.

THIRD NORMAL FORM: A table is in third normal form (3NF). When it is in 2NF and every non-key attribute is functionally dependent on just the primary key.

3.5 DATA FLOW DIAGRAM

Data flow diagram (DFD) is a graphical representation of the “flow” of data through an information system, modelling its process aspects. A DFD is often used as a preliminary step to create an overview of the system without going into great detail, which can later be elaborated. A DED shows what kind of information will be input to and output from the system, how the data will advance through the system, and where the data will be stored.

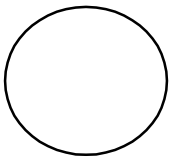
DFD is a designing tool used in the top-down approach to system Design. This context level DFD is next “exploded “, to produce a Level 1 DFD that shows some of the detail of the system being modelled. The Level 1 DFD shows how the system is divided into sub-systems (processes), each of which deals with one or more of the data flows to or from an external agent, and which together provide all of the functionality of the system as a whole. It also identifies internal data stores that must be present in order for the system to do its job and shows the flow of data between the various parts of the system.

- Function- An activity or a function that is performed for some specific reason; can be manual or computerized; ultimately each process should perform only one activity.
- Data Store- collection of data that is permanently stored.
- External Entity- A person, organization or system that is external to the system but interact with it.
- Data Flow- Single piece of data or logical collection of information like a bill.

The following are some DFD symbols used in the project.

- Rectangle: - It defines a source or destination of system data.

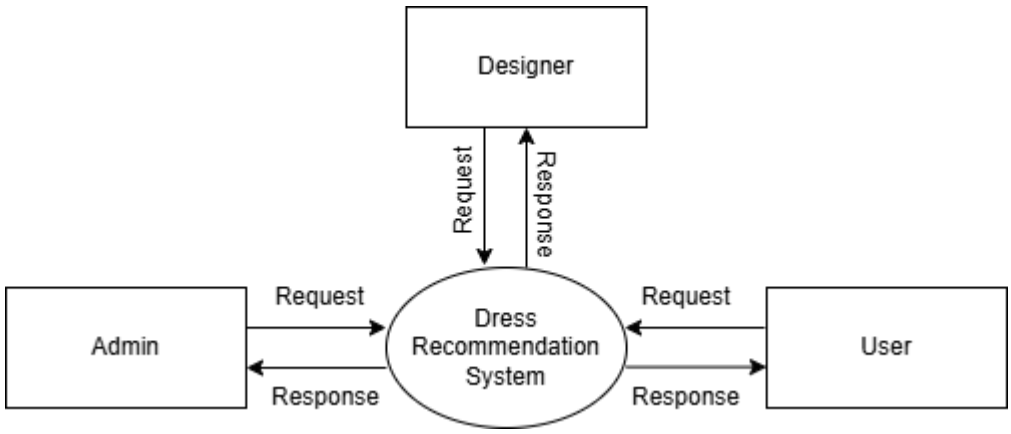
- Circle: - It represents a process that transforms incoming data flow into outgoing data flow.



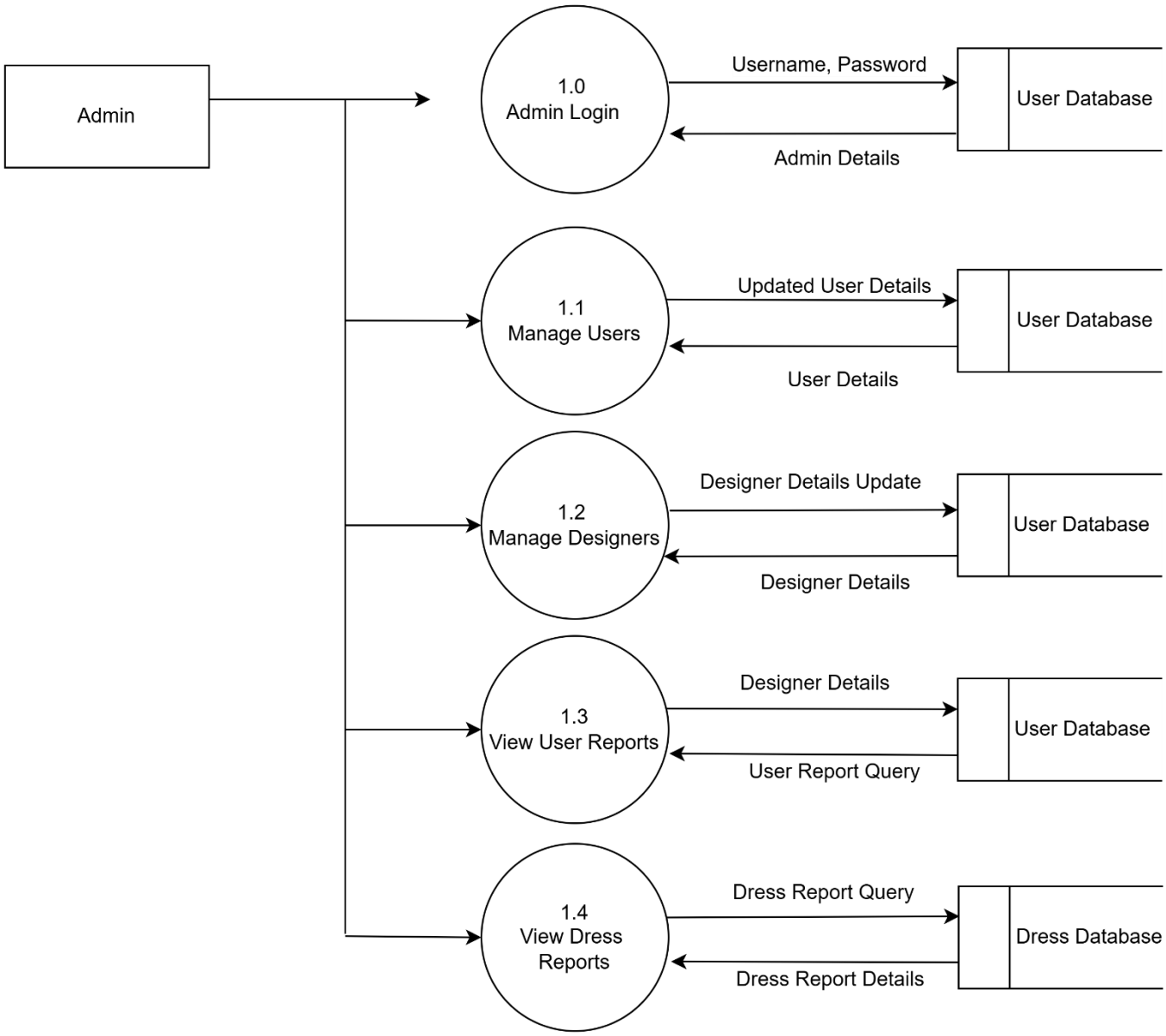
- Arrow: - It defines data flow. It is a pipeline through which information flows.

- Open rectangle: - It is used to store data or a temporary repository of data.

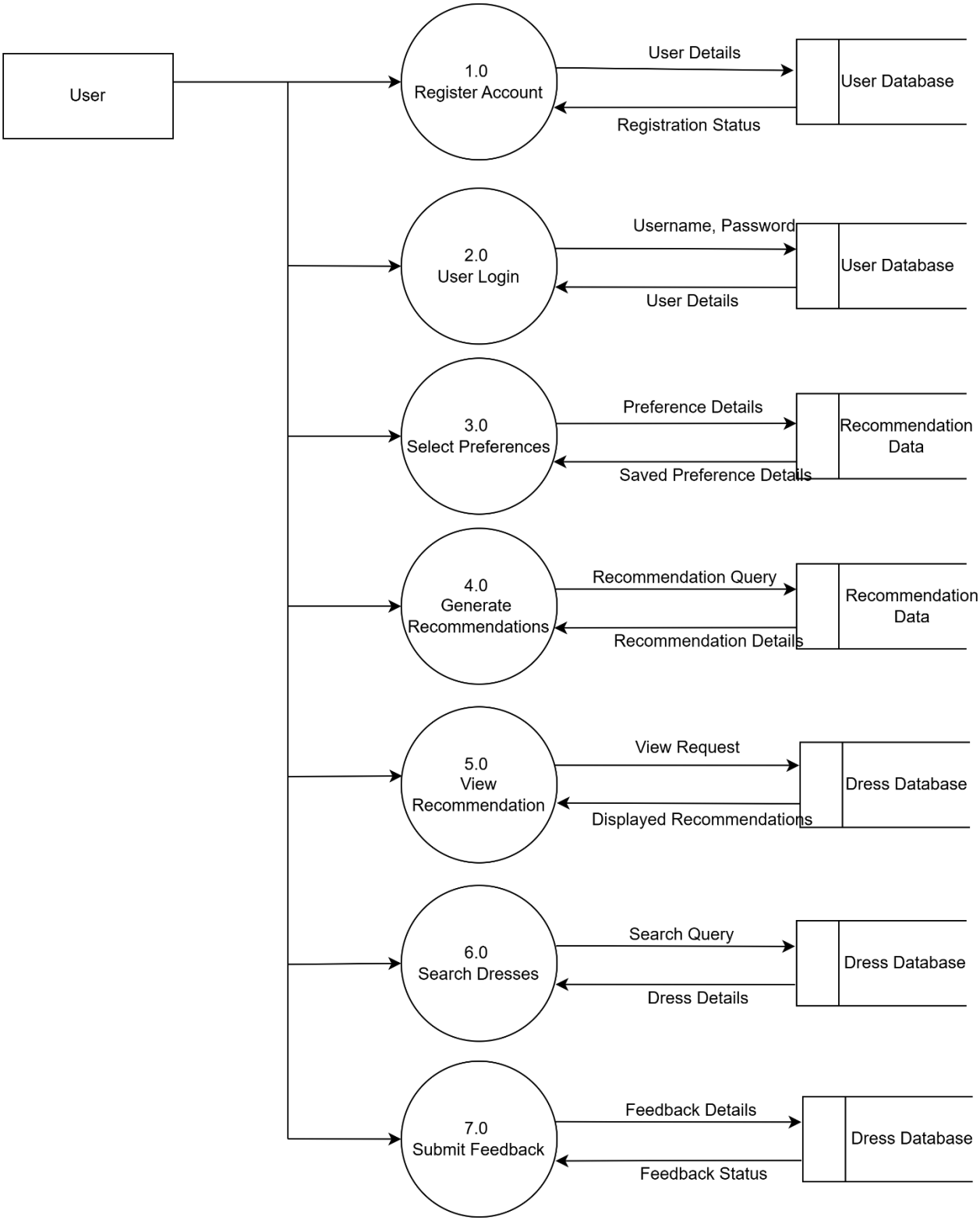
DFD LEVEL- 0



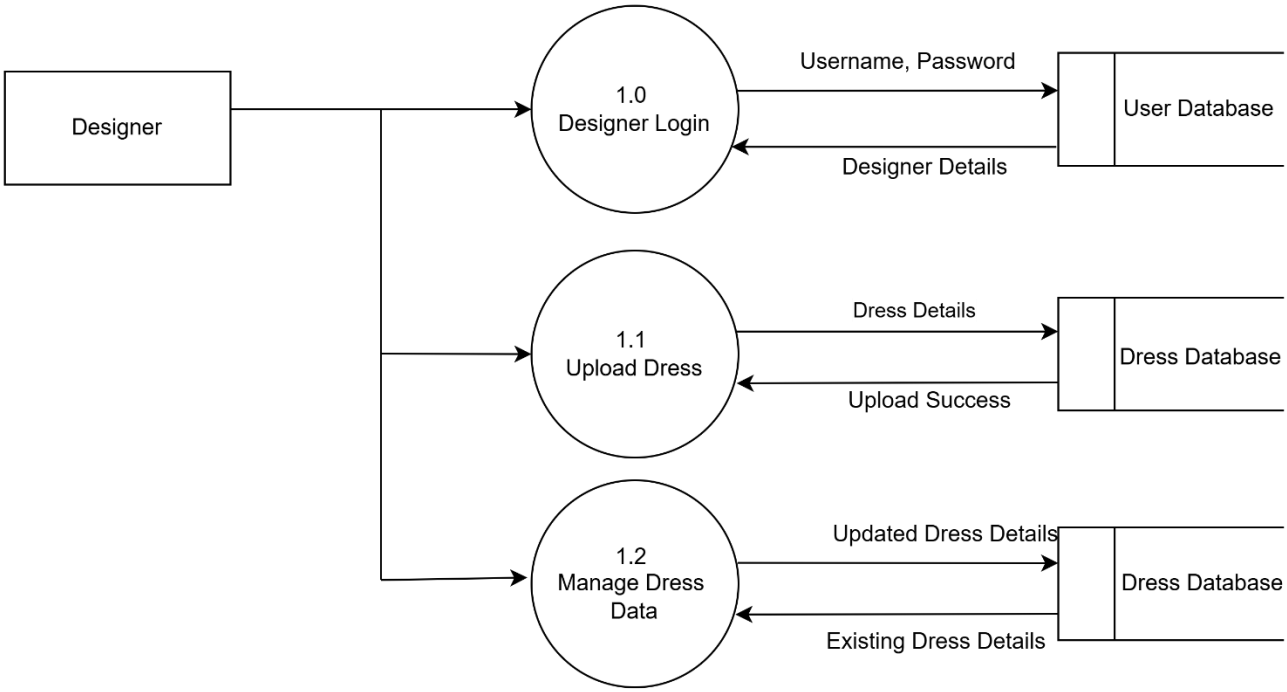
DFD LEVEL - 1 ADMIN



DFD LEVEL - 1 USER



DFD LEVEL - 1 DESIGNER MODULE



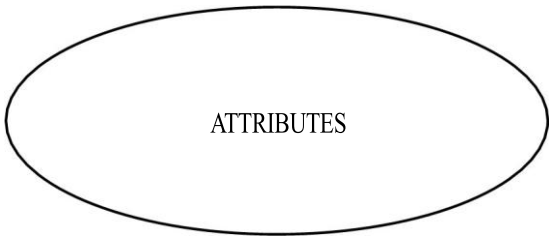
3.6 ENTITY RELATIONSHIP DIAGRAM

ER Diagram stands for Entity Relationship Diagram, also known as ERD is a diagram that displays the relationship of entity sets stored in a database. In other words, ER diagrams help to explain the logical structure of databases. ER diagrams are created based on three basic concepts: entities, attributes and relationships.

ER Diagrams contain different symbols that use rectangles to represent entities, ovals to define attributes and diamond shapes to represent relationships.

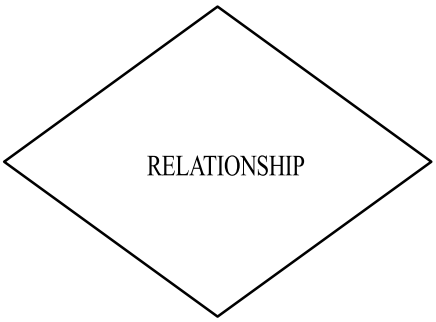
ATTRIBUTES:

Attributes are the properties which define the entity type. In ER diagram, attribute is represented by an oval.



RELATIONSHIP:

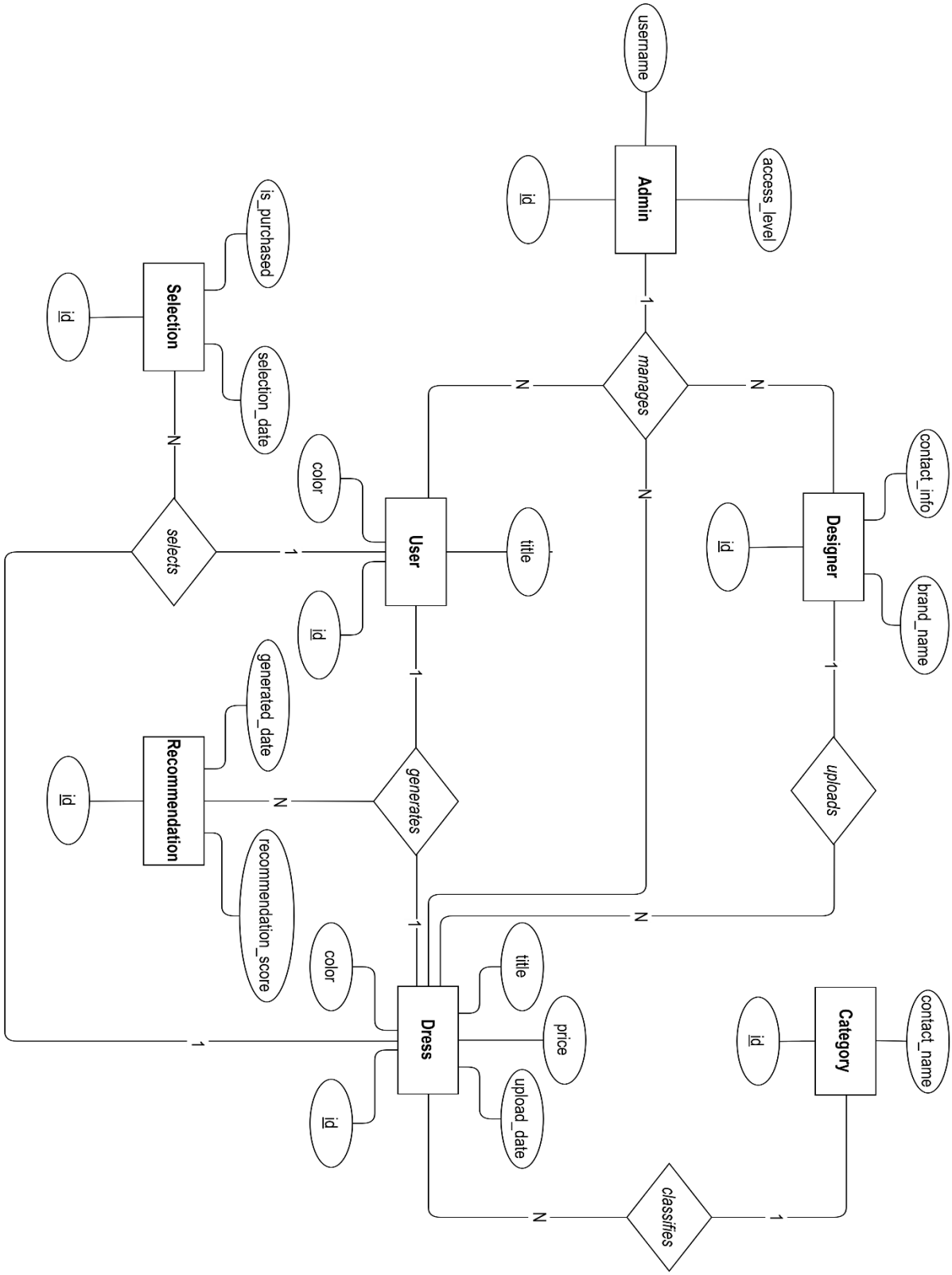
A relationship type represents the association between entity types. In ER diagram, relationship type is represented by a diamond.



ENTITY:

An entity is an object or component of data. An entity is represented as rectangle in an ER diagram.

ER-DIAGRAM


contact_info
brand_name
Designer
id
uploads
1
N
contact_name
Category
id
classifies
1
N
Admin
id
access_level
username
manages
1
N
N
User
title
color
id
generates
1
N
Dress
title
price
upload_date
color
id
selects
1
N
1
Selection
id
is_purchased
selection_date
Recommendation
id
generated_date
recommendation_score

3.7 USE CASE DIAGRAM

A Use Case Diagram is a graphical depiction of a user's possible interactions with a system. A use case diagram shows various use cases and different types of users the system has and will often be accompanied by other types of diagrams as well. The use cases are represented by either circles or ellipses.

STEPS FOR USE CASE DIAGRAM

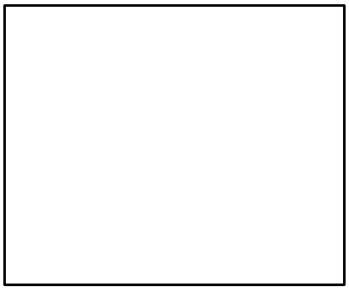
- Identify who is going to be using the website.
- Pick one of those users.
- Define what that user wants to do on the site. Each thing the use does on the site becomes a use case.
- For each use case, decide on the normal course of events when that user is using the site.
- Describe the basic course in the description for the use case. Describe it in terms of what the user does and what the system does in response that the user should be aware of.
- When the basic course is described, consider alternate courses of events and add those to "extend" the use case.
- Look for commonalities among the use cases. Extract these and note them as common course use cases.
- Repeat the steps 2 through 7 for all other users.

Actor:

The users that interact with a system. An actor can be a person, an organization, or an outside system that interacts with your application or system. They must be external objects that produce or consume data.

System:

A box that sets a system scope to use cases. All use cases outside the box would be considered outside the scope of that system.

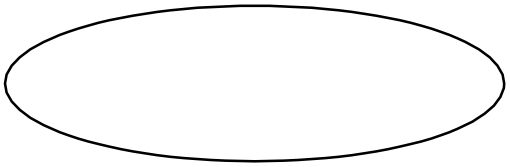


Association:

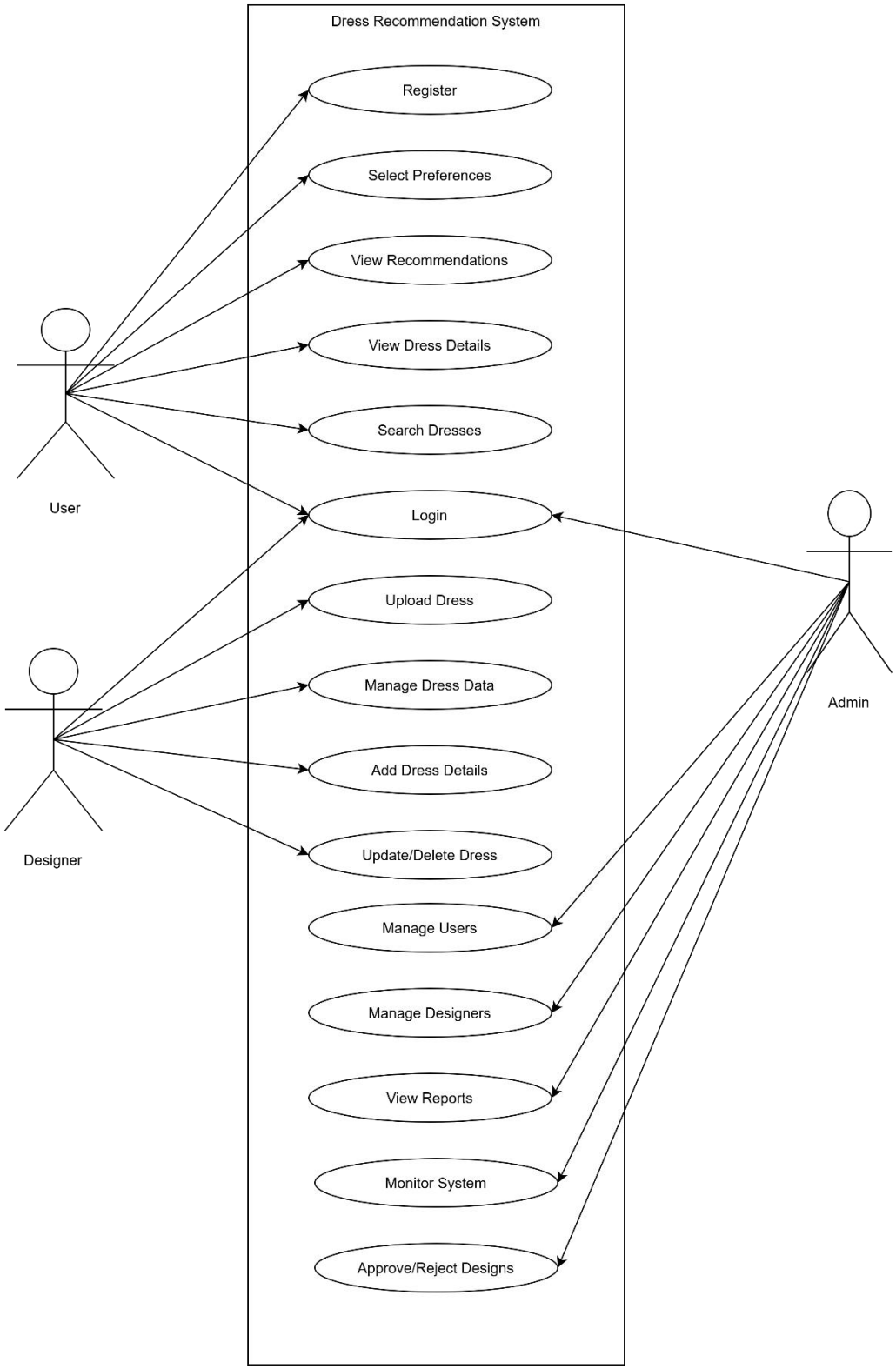
A line between actors and use cases. In complex diagrams, it is important to know which actors are associated with which use cases.

Use cases:

Horizontally shaped ovals that represent the different uses that a user might have.



USECASE DIAGRAM



4. SYSTEM DEVELOPMENT

System Development

System development is the process of designing, building, testing, and implementing the Dress Recommendation System. This phase focuses on converting the system requirements into a working application by following a structured development approach. It ensures that all modules such as user management, recommendation engine, designer dashboard, and admin dashboard are developed properly and integrated into a single platform. The development process follows a step-by-step methodology to ensure that the system functions efficiently and meets user requirements.

4.1 MODULE DESCRIPTION

The Dress Recommendation System is divided into different modules to ensure smooth functioning and easy management of the platform. Each module performs specific tasks and works together to provide personalized recommendations and efficient system operation. The main modules of the system are User Module, Designer Module, Admin Module, and Recommendation Module.

- **USER MODULE:** The User Module allows users to register, log in, and manage their profiles. Users can enter personal details such as body measurements, skin tone, preferred colors, and occasion. Based on this information, the system provides suitable dress recommendations. Users can also browse dress templates, customize designs, and rate the dresses available on the platform.

- **DESIGNER MODULE:** The Designer Module enables designers to upload new dress templates and manage existing designs. Designers can update dress collections regularly to keep the platform updated with the latest fashion trends. This module helps in maintaining a variety of dress templates for users to explore and customize.

- **ADMIN MODULE:** The Admin Module is responsible for managing the overall system. The admin can monitor users and designers, manage dress templates, and ensure smooth functioning of the platform. This module helps in maintaining system security and data management.

- **RECOMMENDATION MODULE:** The Recommendation Module is the core part of the system. It analyzes user input such as body type, skin tone, preferences, and occasion to generate personalized dress suggestions. This module helps users find suitable outfits quickly and improves the overall user experience.

5. SYSTEM IMPLEMENTATION

5.1 TESTING

Software testing is the process of identifying errors in the application that may lead to faults or failure of the entire system. The Dress Recommendation System must pass different testing conditions to ensure that it works correctly and meets user requirements. Testing helps in improving the quality, reliability, and performance of the system by verifying that all modules function properly and provide accurate results.

Various types of testing are performed to ensure the proper functioning of the system. Unit testing is conducted to verify the individual modules such as user registration, login, recommendation engine, and customization features. Integration testing is performed to ensure that all modules work together without errors. System testing checks the overall functionality of the complete application, and user acceptance testing ensures that the system meets the expectations of users, designers, and administrators. Through proper testing, the Dress Recommendation System becomes reliable, secure, and ready for real-world use.

Unit testing

Unit testing is the process of testing individual modules of the Dress Recommendation System to ensure that each component functions correctly. In this phase, every module is tested separately to identify errors at an early stage of development. This helps developers fix issues quickly and improves the overall quality and reliability of the system.

In the Dress Recommendation System, unit testing is performed on modules such as user registration and login, profile management, recommendation engine, dress customization, and dashboard functionalities. Each module is tested using different input values to verify that it produces the expected output. By performing unit testing, errors can be detected and corrected before integrating the modules, ensuring that the system works smoothly and efficiently.

Integration testing

INTEGRATION TESTING

Integration testing is performed to verify that all modules of the Dress Recommendation System work together correctly after unit testing. This testing ensures smooth communication and data

flow between modules such as the user module, recommendation engine, designer module, and admin module. It checks whether user details are properly passed to the recommendation engine, whether uploaded dress templates are correctly displayed, and whether all system components function as a single integrated application without errors.

Systems testing

System testing is carried out to verify the complete Dress Recommendation System as a whole. It ensures that all modules work together properly and meet the system requirements. This testing helps identify any remaining errors and confirms that the application is ready for real-world use.

Acceptance testing

ACCEPTANCE TESTING

Acceptance testing is performed to verify that the Dress Recommendation System meets the requirements and expectations of its users. In this stage, the system is evaluated from the user's perspective to ensure that all features such as registration, login, recommendation generation, dress customization, and rating work properly. This testing helps confirm that the system provides accurate results and is easy to use.

This phase involves users interacting with the system in a real-time environment to check whether it satisfies their needs and requirements. Any final changes or improvements are made based on user feedback. Once the system successfully passes acceptance testing, it is considered ready for deployment and real-world use.

REGRESSION TESTING

Regression testing is carried out to ensure that new updates or changes in the Dress Recommendation System do not affect the existing functionalities. After adding new features or fixing bugs, the system is tested again to verify that all previously working modules continue to function properly.

5.2 VALIDATION CHECKS

A validation check ascertains that the value (or data) input into a computer is valid. Validation checks are performed automatically by computer to ensure that entered data is correct and reasonable. Validation means check the input submitted by the user. There are two types of validation are available in PHP. They are as follows –

Client-Side Validation − Validation is performed on the client machine web browsers.

Server-Side Validation – After submitted by data, the data has sent to a server and perform validation checks in server machine.

Server-Side Validation

In the Server-Side Validation, the input submitted by the user is being sent to the server and validated using one of server-side scripting languages such as ASP.Net, PHP etc. After the validation process on the Server Side, the feedback is sent back to the client by a new dynamically generated web page. It is better to validate user input on Server Side because you can protect against the malicious users, who can easily bypass your Client-Side scripting language and submit dangerous input to the server.

Client-Side Validation

In the Client-Side Validation, you can provide a better user experience by responding quickly at the browser level. When you perform a Client-Side Validation, all the user inputs validated in the user's browser itself. Client-Side validation does not require a round trip to the server, so the network traffic which will help your server perform better. This type of validation is done on the browser side using script languages such as JavaScript, VBScript or HTML5 attributes.

TEST CASE

Test case	Form	Module	Description	Test result	Pass/Fail
1.	Login	Admin, User, Designer	Check results on entering valid username and password	Login successful	Pass
2.	Login	Admin, User, Designer	Check results on entering invalid username and password	Print error of "Invalid username and password."	Pass
3.	Login	Admin, User, Designer	Check response when username is empty and login button is pressed	Login should not proceed	Pass
4.	Manage User	Admin	Add/Update/Delete users	User management successful	Pass
5.	Upload Dress	Designer	Upload new dress details (image, category, style)	Dress uploaded successfully	Pass
6.	Generate Recommendation	System	Generate dress recommendations based on user preferences	Recommendations generated successfully	Pass
7.	Search Dresses	User	Search dresses using keywords or filters	Relevant dresses displayed correctly	Pass
8.	View Dress Details	User	View detailed information of selected dress	Dress details displayed correctly	Pass
9.	Manage Designers	Admin	Approve/Reject designers	Designer status updated successfully	Pass
10	View Reports	Admin	Check system reports and analytics	Reports displayed correctly	Pass

5.3 SYSTEM IMPLEMENTATION

SYSTEM IMPLEMENTATION

System implementation involves placing the Dress Recommendation System into operation and providing users, designers, and administrators with the necessary guidance to use and maintain the system effectively. This phase includes all activities required to convert from the existing manual or general online shopping approach to the new personalized recommendation system. Proper implementation is essential to ensure that the system works reliably and meets user requirements.
The different methods used for system implementation are:

- **Parallel Approach:** The existing method of selecting dresses and the new Dress Recommendation System are used together for a certain period. This helps users become familiar with the new system while ensuring a smooth transition.
- **Direct Cutover Method:** The old method is completely replaced by the new Dress Recommendation System in a single step. This approach is faster but requires careful planning to avoid system issues.
- **Pilot Approach**: The system is first introduced to a small group of users. Based on their feedback, improvements are made before implementing the system for all users.
- **Phase-in Method:** The system is introduced gradually by implementing different modules step by step. This reduces risk and allows users to adapt to the new system easily.

5.4 SECURITY

SYSTEM SECURITY

System security refers to the protection of computer-based resources such as hardware, software, data, and procedures from unauthorized access, misuse, and natural threats. In the Dress Recommendation System, security plays an important role in protecting user information such as personal details, preferences, and login credentials. Proper security measures ensure that the system operates safely and reliably.
System security in the proposed system focuses on the following aspects:

- **Confidentiality:** User data such as personal details, measurements, and preferences are kept secure and accessible only to authorized users. Authentication and login mechanisms help protect sensitive information.
- **Integrity:** The system ensures that data stored in the database is accurate and protected from unauthorized modification or deletion. Proper validation and access control help maintain data integrity.
- **Privacy:** The system protects user information from being disclosed to unauthorized persons. Secure data storage and restricted access help maintain user privacy.
- **Disaster Protection:** Backup and recovery mechanisms help protect the system from data loss due to technical failures or unexpected events.

Data Security ensures that all user and system data are protected from loss, disclosure, modification, and destruction. **System Integrity** ensures the proper functioning of hardware and software, along with protection from external threats such as unauthorized access and network attacks.

.

6. SYSTEM MAINTENANCE AND FUTURE ENHANCEMENT

6.1 SYSTEM MAINTENANCE

SYSTEM MAINTENANCE

System maintenance refers to the process of updating, improving, and fixing the Dress Recommendation System after implementation. It ensures that the system continues to function properly and remains efficient over time. Maintenance includes correcting errors, updating dress templates, improving performance, and adding new features when required.

Regular maintenance helps keep the system secure, reliable, and up to date. It also ensures that user requirements and technological changes are handled effectively, allowing the system to operate smoothly in the long term.

Maintenance Types

System maintenance can be classified into four types:

- Corrective Maintenance
- Adaptive Maintenance
- Perfective Maintenance
- Preventive Maintenance

MAINTENANCE TYPES

Corrective Maintenance

- Corrective maintenance focuses on identifying and fixing errors that appear after the system is deployed.
- It ensures that bugs and technical issues are resolved quickly.
- This includes fixing login errors, database issues, or recommendation mismatches.
- Timely correction improves system reliability and performance.
- It helps maintain smooth functioning of the application.
- This type of maintenance increases user satisfaction and trust.

Adaptive Maintenance

- Adaptive maintenance is carried out when the system needs updates to match new technologies.
- It helps the system remain compatible with new devices, browsers, and software updates.
- Changes in user requirements are also handled through this maintenance.
- It ensures the system continues to work efficiently in a changing environment.
- This type of maintenance keeps the system up to date.
- It supports long-term usability and flexibility.

Perfective Maintenance

- Perfective maintenance aims to improve system performance and usability.
- It includes adding new features and enhancing existing functionalities.
- User feedback is used to upgrade the system.
- Improvements in the recommendation engine are made in this phase.
- It helps make the system more efficient and user-friendly.
- This maintenance enhances overall user experience.

Preventive Maintenance

- Preventive maintenance focuses on avoiding future system problems.
- Regular system monitoring and performance checks are carried out.
- Security updates and backups are performed regularly.
- Database optimization is done to improve efficiency.
- It reduces the risk of system failures.
- This ensures long-term stability and reliability of the system.

6.2 FUTURE ENHANCEMENTS

FUTURE ENHANCEMENTS

While the current Dress Recommendation System provides a strong foundation for personalized outfit suggestions and customization, there is significant scope for future improvements and expansion. Some of the key enhancements include:

1. **Integration with Advanced AI Techniques**

- Incorporate advanced machine learning algorithms to provide more accurate and intelligent dress recommendations based on user behavior and fashion trends.

2. **Mobile Application Development**

- Develop a user-friendly mobile application that allows users to access dress recommendations, customization features, and templates anytime and anywhere.

3. **Virtual Try-On Feature**

- Implement augmented reality (AR) technology to allow users to virtually try on dresses and visualize how they look before selecting them.

4. **Real-Time Designer Interaction**

- Enable real-time communication between users and designers for better customization and feedback.

5. **Advanced Analytics Dashboard**

- Create a dashboard for admin and designers to analyze user preferences, popular designs, and fashion trends.

6. **Integration with Online Shopping Platforms**

- Connect the system with e-commerce platforms to allow users to directly purchase recommended dresses.

7. **Cloud Integration for Scalability**

- Deploy the system on cloud platforms to support remote access, secure storage, and large-scale usage.

8. **Enhanced Security and Privacy**

- Implement advanced data protection techniques to secure user information and ensure privacy.

9. **Continuous Learning and Dataset Expansion**

- Continuously update the system with new fashion data to improve recommendation accuracy and system performance.

These enhancements will make the Dress Recommendation System more intelligent, scalable, and suitable for future technological advancements.

7. CONCLUSION

7.1 CONCLUSION

In today's digital environment, providing personalized fashion recommendations has become a key factor in enhancing user experience on online shopping platforms. This project demonstrates the potential of combining Artificial Intelligence and intelligent recommendation techniques to create a system capable of suggesting outfits tailored to individual preferences, body types, and style choices.

By analyzing physical attributes, style history, and user interactions, the system offers an efficient and personalized way to select dresses while reducing confusion and dissatisfaction. The integration of features like dress self-customization, star ratings, separate dashboards for designers and admins, and return policies ensures smooth interaction, controlled collaboration, and improved user engagement.

Although there are certain limitations—such as dependency on accurate user input, computational resources, and system complexity—the proposed system establishes a strong foundation for future advancements in fashion technology. With further enhancements like virtual try-ons, AI-driven trend analysis, and predictive style suggestions, the system can evolve into a highly reliable and fully immersive fashion recommendation platform.

Overall, this project highlights the importance of leveraging intelligent technologies to personalize shopping experiences and provides a step toward smarter, more interactive, and satisfying digital fashion solutions.

8. APPENDIX

8.1APPENDIX A. TABLE DESIGNS

1. USER TABLE

FIELD	TYPE	LENGTH	CONSTRAINTS
user_id	INT	11	Primary Key, Auto Increment
username	VARCHAR	50	Not Null, Unique
email	VARCHAR	100	Not Null, Unique
password	VARCHAR	255	Not Null
phone	VARCHAR	20	Not Null
address	VARCHAR	255	Default NULL
created_at	DATETIME	-	Default CURRENT_TIMESTAMP
status	BOOLEAN	1	Default 1

2. DESIGNER TABLE

FIELD	TYPE	LENGTH	CONSTRAINTS
designer_id	INT	11	Primary Key, Auto Increment
name	VARCHAR	100	Not Null
email	VARCHAR	100	Not Null, Unique
password	VARCHAR	255	Not Null
phone	VARCHAR	20	Not Null
portfolio_url	VARCHAR	255	Default NULL
experience	INT	2	Default 0
is_approved	BOOLEAN	1	Default 0
created_at	DATETIME	-	Default CURRENT_TIMESTAMP

3. ADMIN TABLE

FIELD	TYPE	LENGTH	CONSTRAINTS
category_id	INT	11	Primary Key, Auto Increment
category_name	VARCHAR	100	Not Null, Unique
description	VARCHAR	255	Default NULL
status	BOOLEAN	1	Default 1
timestamp	datetime	-	Not Null

4. CATEGORY TABLE

FIELD	TYPE	LENGTH	CONSTRAINTS
dress_id	INT	11	Primary Key, Auto Increment
designer_id	INT	11	Foreign Key → DESIGNER(designer_id), Not Null
category_id	INT	11	Foreign Key → CATEGORY(category_id), Not Null
name	VARCHAR	150	Not Null
description	TEXT	-	Not Null
price	DECIMAL	10,2	Not Null
color	VARCHAR	30	Not Null
image_url	VARCHAR	255	Default NULL
stock_qty	INT	5	Default 0
created_at	DATETIME	-	Default CURRENT_TIMESTAMP

5. DRESS TABLE

FIELD	TYPE	LENGTH	CONSTRAINTS
dress_id	INT	11	Primary Key, Auto Increment
designer_id	INT	11	Foreign Key → DESIGNER(designer_id), Not Null
category_id	INT	11	Foreign Key → CATEGORY(category_id), Not Null
name	VARCHAR	150	Not Null
description	TEXT	-	Not Null
price	DECIMAL	10,2	Not Null
color	VARCHAR	30	Not Null
material	VARCHAR	50	Not Null
image_url	VARCHAR	255	Default NULL
stock_qty	INT	5	Default 0
created_at	DATETIME	-	Default CURRENT_TIMESTAMP

6. RECOMMENDATION TABLE

FIELD	TYPE	LENGTH	CONSTRAINTS
recommendation_id	INT	11	Primary Key, Auto Increment
user_id	INT	11	Foreign Key → USER(user_id), Not Null
dress_id	INT	11	Foreign Key → DRESS(dress_id), Not Null
score	DECIMAL	5,2	Default NULL

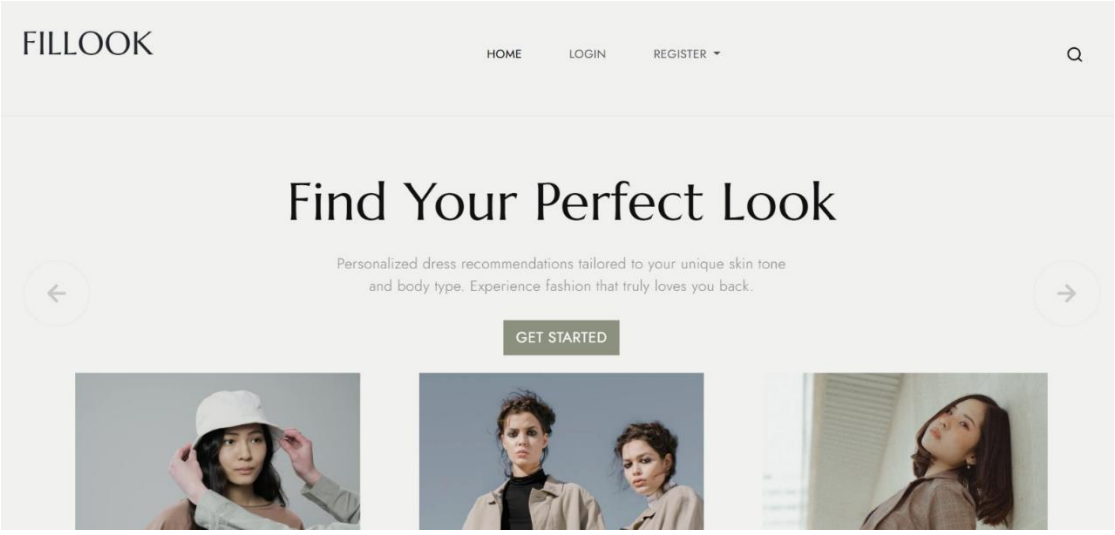
rationale	VARCHAR	255	Default NULL
created_at	DATETIME	-	Default CURRENT_TIMESTAMP

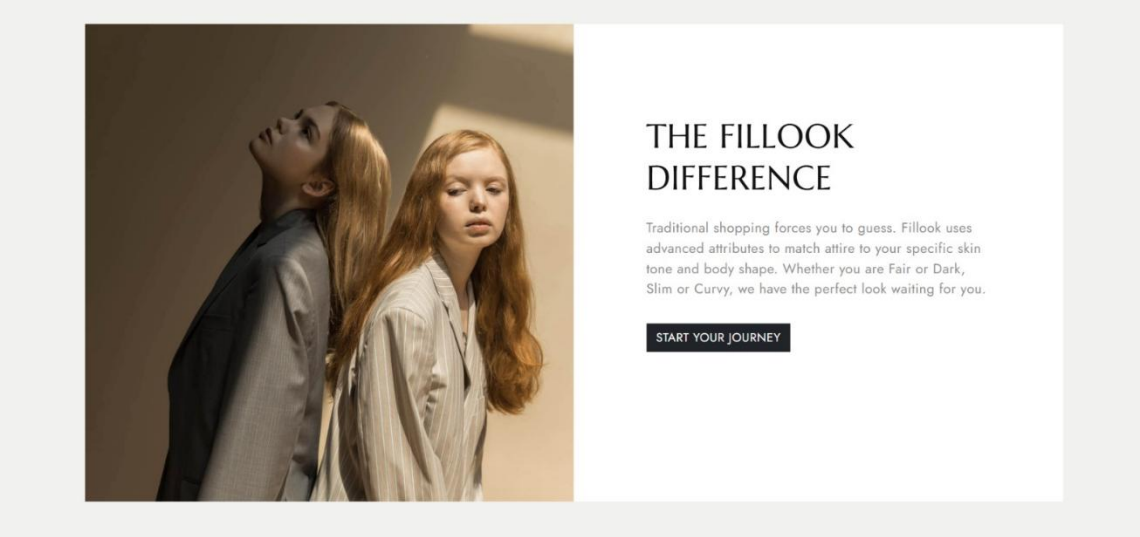
7. SELECTION TABLE

FIELD	TYPE	LENGTH	CONSTRAINTS
selection_id	INT	11	Primary Key, Auto Increment
user_id	INT	11	Foreign Key → USER(user_id), Not Null
dress_id	INT	11	Foreign Key → DRESS(dress_id), Not Null
selected_size	VARCHAR	10	Not Null
status	VARCHAR	20	Default 'Pending'
selected_at	DATETIME	-	Default CURRENT_TIMESTAMP

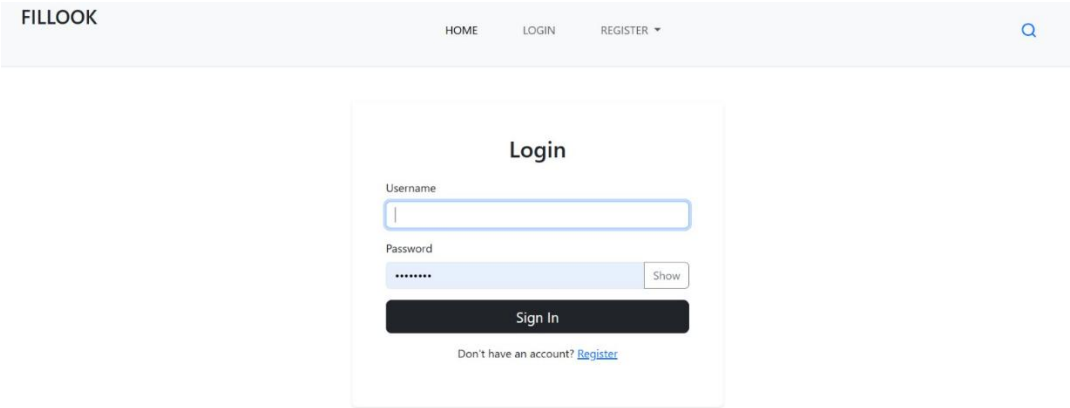
8.2 APPENDIX B. SAMPLE INPUT AND OUTPUT DESIGN

1. HOME PAGE

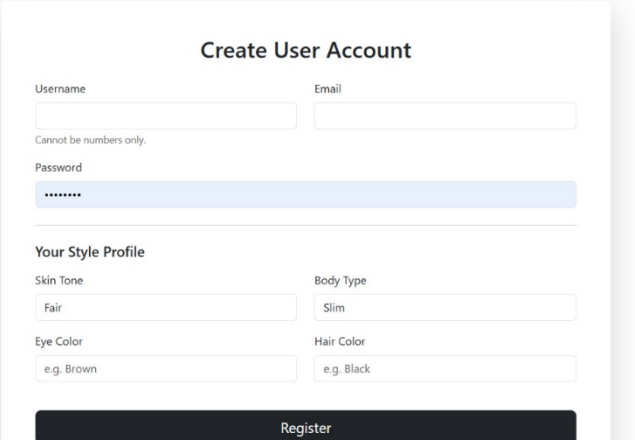




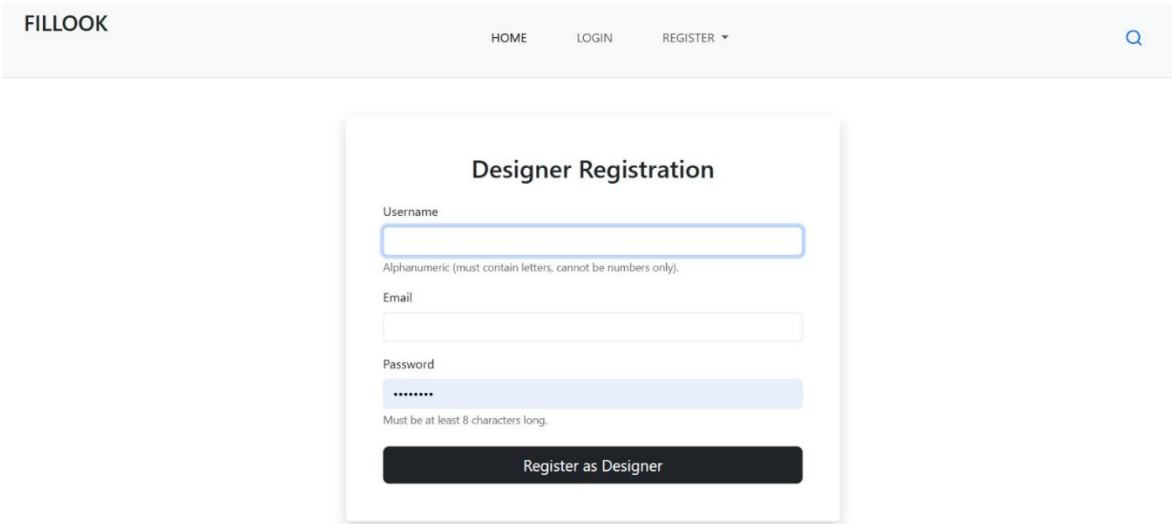
2. LOGIN PAGE



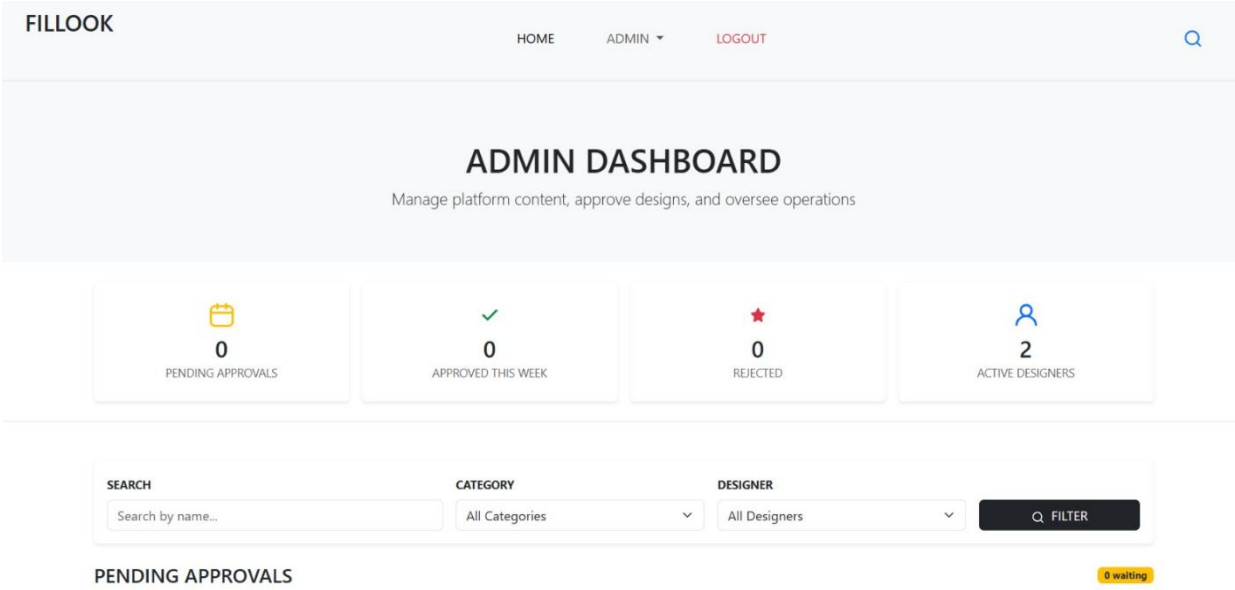
3. USER REGISTER PAGE



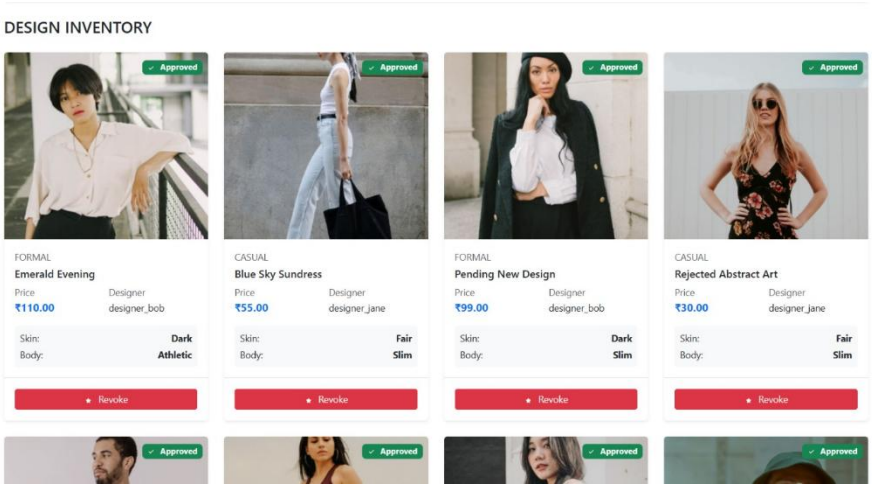
4. DESIGNER REGISTER PAGE



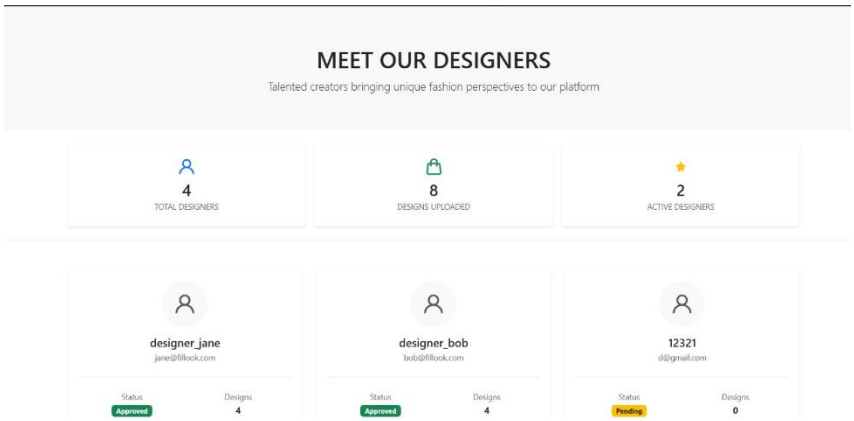
5. ADMIN DASHBOARD PAGE



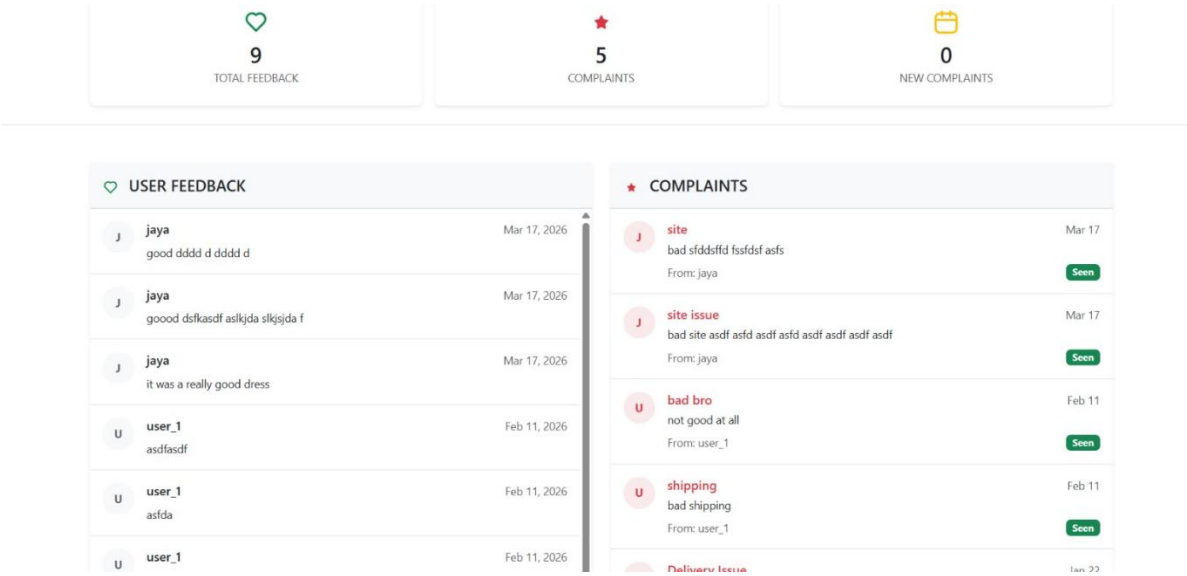
6. ADMIN DRESS MANAGEMENT AND PERMISSION CONTROL PAGE



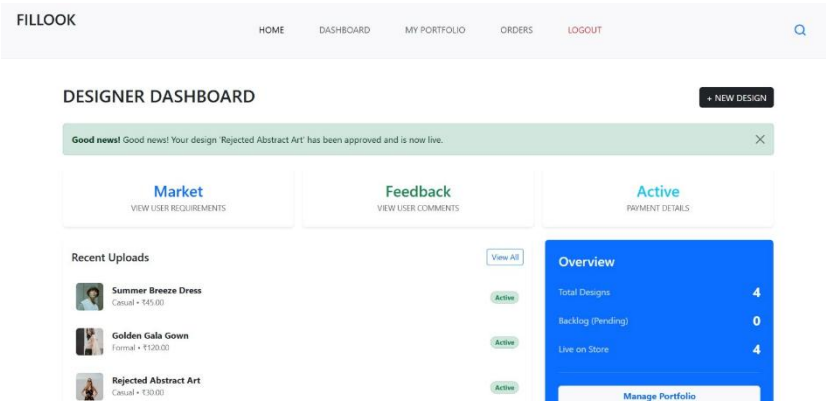
7. ADMIN DESIGNER VIEW PAGE



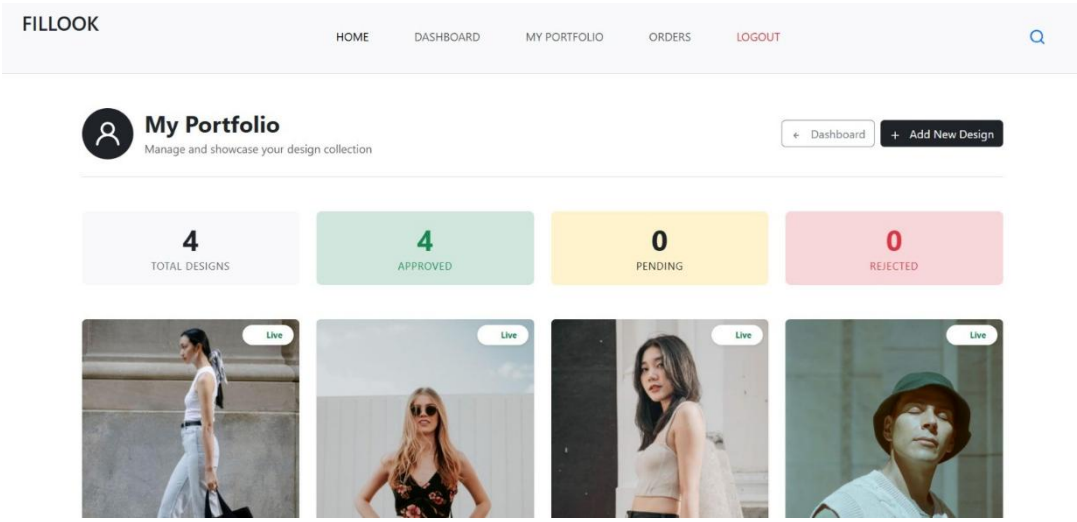
8. USER FEEDBACK AND COMPLAINT VIEW PAGE



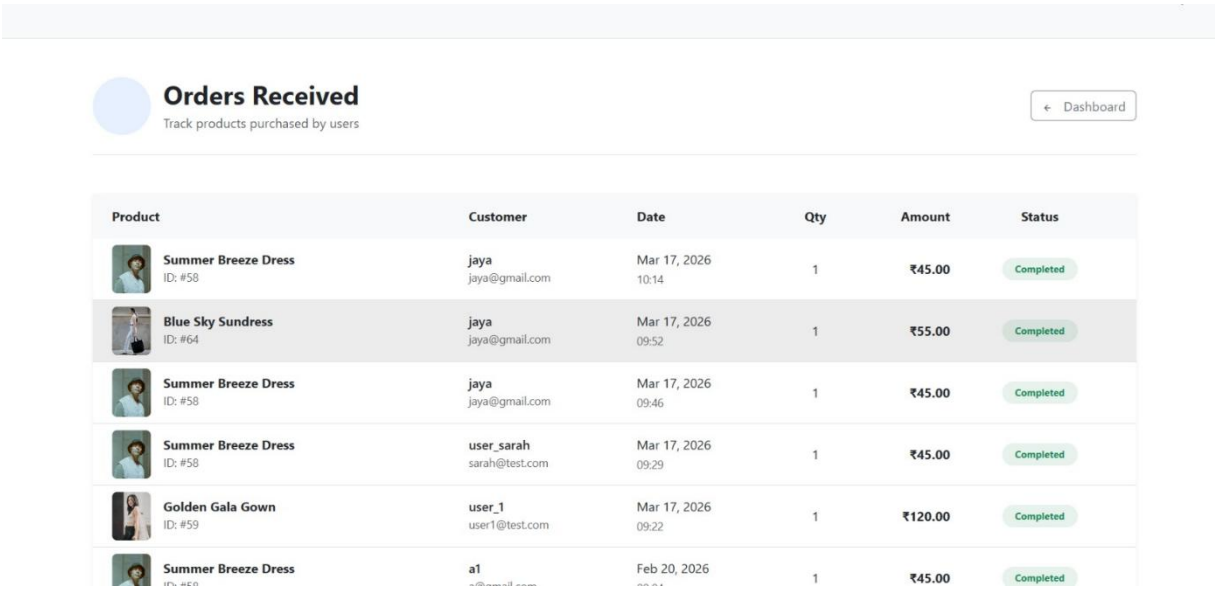
9. DESIGNER DASHBOARD PAGE



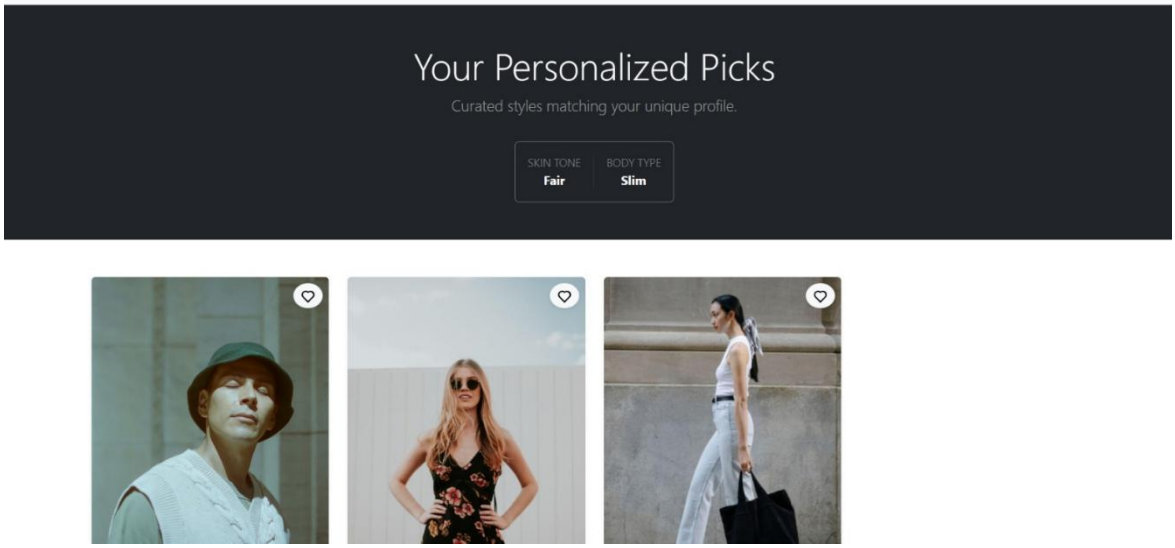
10. DESIGNER PORTFOLIO PAGE



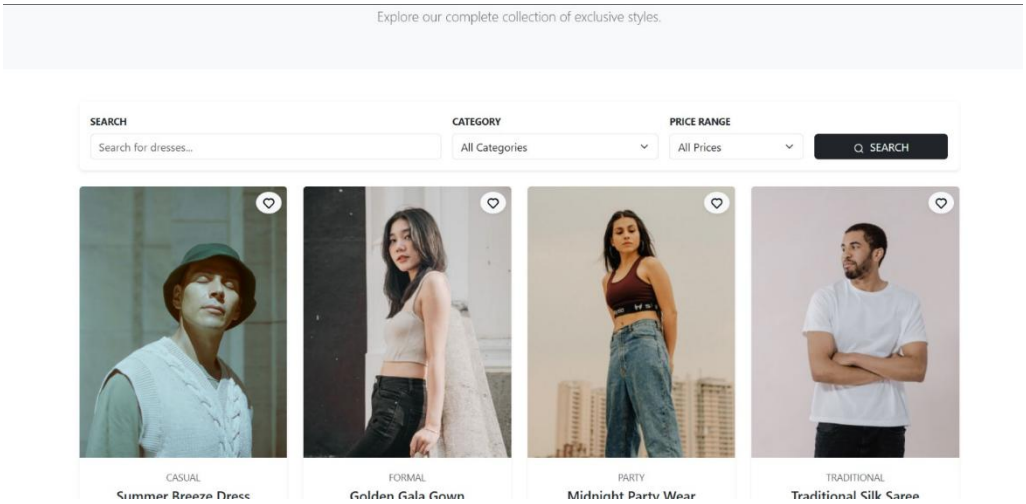
11. ORDERS PAGE



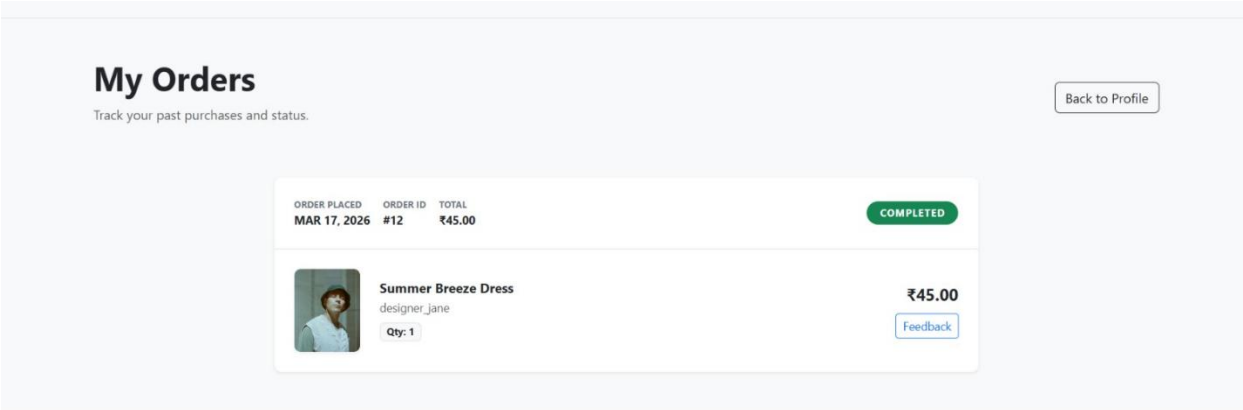
12. USER HOME PAGE



13. USER DRESS SEARCH PAGE



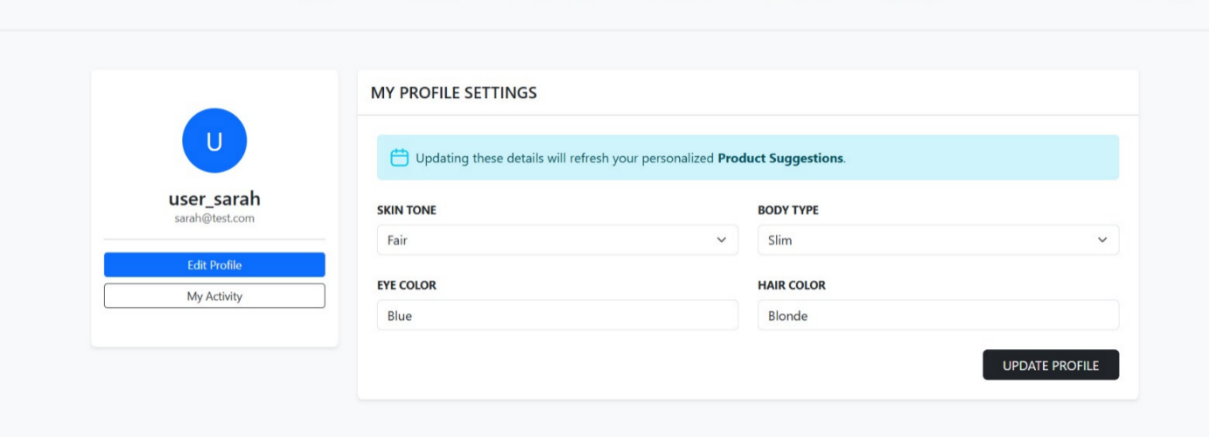
14. USER ORDERS PAGE



Fillook

Your personalized dress recommendation engine.

15. USER MY PROFILE PAGE



Fillook

Your personalized dress recommendation engine.

8.3 APPENDIX C. SOURCE CODE

1. LOGIN.HTML

```
{% extends 'base.html' %}
{% load static %}

{% block content %}
<section class="login-section py-5">
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-md-6 col-lg-5">
        <div class="card shadow-sm border-0">
          <div class="card-body p-5">
            <h2 class="text-center mb-4">Login</h2>

            {% if error %}
            <div class="alert alert-danger">{{ error }}</div>
            {% endif %}

            <form method="POST">
              {% csrf_token %}
              <div class="mb-3">
                <label for="username" class="form-label">Username</label>
                <input  type="text"  class="form-control"  id="username"  name="username"
required>
              </div>
              <div class="mb-3">
                <label for="password" class="form-label">Password</label>
                <div class="input-group">
                  <input  type="password"  class="form-control"  id="password"
name="password" required>
                  <button  class="btn  btn-outline-secondary"  type="button"
onclick="togglePassword()">
                    Show
                  </button>
                </div>
              </div>
              <div class="d-grid gap-2">
                <button type="submit" class="btn btn-dark btn-lg">Sign In</button>
              </div>
```

```
                    </form>
                    <div class="text-center mt-3">
                        <p>Don't have an account? <a href="{% url 'register_user' %}">Register</a></p>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>

<script>
function togglePassword() {
  var x = document.getElementById("password");
  if (x.type === "password") {
    x.type = "text";
  } else {
    x.type = "password";
  }
}
</script>
{% endblock %}
```

2. REGISTER.HTML

```
 {% extends 'base.html' %}
{% load static %}

{% block content %}
<section class="register-section py-5">
   <div class="container">
      <div class="row justify-content-center">
         <div class="col-lg-6">
            <div class="card shadow border-0">
               <div class="card-body p-5">
                  <h2 class="text-center mb-4">Designer Registration</h2>
```

```
{% if error %}
  <div class="alert alert-danger">{{ error }}</div>
{% endif %}

<form method="POST">
  {% csrf_token %}
  <div class="mb-3">
    <label class="form-label">Username</label>
    <input type="text" class="form-control" name="username" pattern=".*[a-zA-Z]+.*"
title="Must contain at least one letter" required>
    <div class="form-text">Alphanumeric (must contain letters, cannot be numbers
only).</div>
  </div>
  <div class="mb-3">
    <label class="form-label">Email</label>
    <input type="email" class="form-control" name="email" required>
  </div>
  <div class="mb-3">
    <label class="form-label">Password</label>
    <input type="password" class="form-control" name="password" minlength="8" required>
    <div class="form-text">Must be at least 8 characters long.</div>
  </div>

  <!-- Portfolio input hidden as per updated requirements -->
  <!--
  <div class="mb-3">
    <label class="form-label">Portfolio Link</label>
    <input type="url" class="form-control" name="portfolio" placeholder="https://...">
  </div>
  -->
```

```
                <div class="d-grid gap-2 mt-4">
                    <button type="submit" class="btn btn-dark btn-lg">Register as Designer</button>
                </div>
            </form>
        </div>
    </div>
  </div>
 </div>
</section>
{% endblock %}
```

3. ALLDRESS.HTML

```
{% extends 'base.html' %}

{% block content %}
<!-- Hero Section -->
<section id="admin-hero" class="bg-light py-5">
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-md-10 text-center" data-aos="fade-up">
        <h1 class="section-title text-uppercase mt-4">All Dresses</h1>
        <p class="lead">Complete inventory of all uploaded designs across all statuses</p>
      </div>
    </div>
  </div>
</section>

<!-- Stats Section -->
<section class="py-4 bg-white border-bottom">
  <div class="container">
    <div class="row g-3">
      <div class="col-md-3">
        <div class="card border-0 shadow-sm text-center h-100">
          <div class="card-body py-4">
            <svg width="32" height="32" class="text-primary mb-2"><use
xlink:href="#shopping-bag"></use></svg>
```

```
                <h3 class="mb-0">{{ total_count }}</h3>
                <small class="text-muted text-uppercase">Total Dresses</small>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm text-center h-100">
            <div class="card-body py-4">
                <svg width="32" height="32" class="text-success mb-2"><use
xlink:href="#check"></use></svg>
                <h3 class="mb-0">{{ approved_count }}</h3>
                <small class="text-muted text-uppercase">Approved</small>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm text-center h-100">
            <div class="card-body py-4">
                <svg width="32" height="32" class="text-warning mb-2"><use
xlink:href="#calendar"></use></svg>
                <h3 class="mb-0">{{ pending_count }}</h3>
                <small class="text-muted text-uppercase">Pending</small>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm text-center h-100">
            <div class="card-body py-4">
                <svg width="32" height="32" class="text-danger mb-2"><use
xlink:href="#close"></use></svg>
                <h3 class="mb-0">{{ rejected_count }}</h3>
                <small class="text-muted text-uppercase">Rejected</small>
            </div>
        </div>
    </div>
  </div>
 </div>
</section>

<!-- All Dresses Grid Section -->
<section class="py-5">
```

```
<div class="container">
  <!-- Search and Filter Bar -->
  <div class="card border-0 shadow-sm mb-4" data-aos="fade-up">
    <div class="card-body">
      <form method="GET" action="">
        <div class="row g-3 align-items-end">
          <div class="col-md-4">
            <label class="form-label small text-uppercase fw-bold">Search</label>
            <input type="text" name="search" class="form-control" placeholder="Search by
name..." value="{{ request.GET.search }}">
          </div>
          <div class="col-md-2">
            <label class="form-label small text-uppercase fw-bold">Status</label>
            <select name="status" class="form-select">
              <option value="">All Statuses</option>
              <option value="Approved" {% if request.GET.status == 'Approved'
%}selected{% endif %}>Approved</option>
              <option value="Pending" {% if request.GET.status == 'Pending' %}selected{%
endif %}>Pending</option>
              <option value="Rejected" {% if request.GET.status == 'Rejected'
%}selected{% endif %}>Rejected</option>
            </select>
          </div>
          <div class="col-md-2">
            <label class="form-label small text-uppercase fw-bold">Category</label>
            <select name="category" class="form-select">
              <option value="">All Categories</option>
              <option value="Casual" {% if request.GET.category == 'Casual'
%}selected{% endif %}>Casual</option>
              <option value="Formal" {% if request.GET.category == 'Formal'
%}selected{% endif %}>Formal</option>
              <option value="Party" {% if request.GET.category == 'Party' %}selected{%
endif %}>Party</option>
              <option value="Traditional" {% if request.GET.category == 'Traditional'
%}selected{% endif %}>Traditional</option>
            </select>
          </div>
          <div class="col-md-2">
            <label class="form-label small text-uppercase fw-bold">Designer</label>
            <select name="designer" class="form-select">
              <option value="">All Designers</option>
```

```
                    {% for designer in all_designers %}
                    <option value="{{ designer.id }}" {% if request.GET.designer ==
designer.id|stringformat:"s" %}selected{% endif %}>{{ designer.user.username }}</option>
                    {% endfor %}
                  </select>
                </div>
                <div class="col-md-2">
                  <button type="submit" class="btn btn-dark w-100 text-uppercase">
                    <svg width="16" height="16" class="me-1"><use
xlink:href="#search"></use></svg>
                    Filter
                  </button>
                </div>
              </div>
            </form>
          </div>
        </div>

        <div class="d-flex justify-content-between align-items-center mb-4">
          <h4 class="text-uppercase mb-0">Design Inventory</h4>
          {% if request.GET.search or request.GET.status or request.GET.category or
request.GET.designer %}
          <a href="{% url 'admin_all_dresses' %}" class="btn btn-sm btn-outline-secondary">Clear
Filters</a>
          {% endif %}
        </div>

        {% if dresses %}
        <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 row-cols-xl-4 g-4">
          {% for dress in dresses %}
          <div class="col" data-aos="fade-up">
            <div class="card h-100 shadow-sm border-0 product-card">
              <div class="position-relative overflow-hidden image-zoom-effect">
                {% if dress.image %}
                <img src="{{ dress.image.url }}" class="card-img-top" alt="{{ dress.name }}"
style="height: 280px; object-fit: cover;">
                {% else %}
                <div class="bg-light d-flex align-items-center justify-content-center" style="height:
280px;">
                  <svg width="48" height="48" class="text-muted"><use xlink:href="#shopping-
bag"></use></svg>
```

```
                </div>
                {% endif %}

                <div class="position-absolute top-0 end-0 m-2">
                    {% if dress.status == 'Approved' %}
                    <span class="badge bg-success">
                        <svg width="12" height="12" class="me-1"><use
xlink:href="#check"></use></svg>
                        Approved
                    </span>
                    {% elif dress.status == 'Rejected' %}
                    <span class="badge bg-danger">Rejected</span>
                    {% else %}
                    <span class="badge bg-warning text-dark">Pending</span>
                    {% endif %}
                </div>
            </div>

            <div class="card-body">
                <small class="text-muted text-uppercase d-block mb-1">{{ dress.category
}}</small>
                <h6 class="card-title mb-2">{{ dress.name }}</h6>

                <div class="row g-2 mb-3">
                    <div class="col-6">
                        <small class="text-muted d-block">Price</small>
                        <strong class="text-primary">₹{{ dress.price }}</strong>
                    </div>
                    <div class="col-6">
                        <small class="text-muted d-block">Designer</small>
                        <span class="small">{{ dress.designer.user.username }}</span>
                    </div>
                </div>

                <div class="bg-light p-2 rounded small">
                    <div class="d-flex justify-content-between mb-1">
                        <span>Skin:</span>
                        <strong>{{ dress.suitable_skin_tone }}</strong>
                    </div>
                    <div class="d-flex justify-content-between">
                        <span>Body:</span>
```

```
                <strong>{{ dress.suitable_body_type }}</strong>
              </div>
            </div>
          </div>

          <div class="card-footer bg-white border-top p-3">
            {% if dress.status == 'Pending' %}
            <div class="d-flex gap-2">
              <a href="{% url 'approve_dress' dress.id %}" class="btn btn-sm btn-success flex-
grow-1">
                <svg width="14" height="14" class="me-1"><use
xlink:href="#check"></use></svg>
                Approve
              </a>
              <a href="{% url 'reject_dress' dress.id %}" class="btn btn-sm btn-outline-danger
flex-grow-1">
                Reject
              </a>
            </div>
            {% elif dress.status == 'Approved' %}
            <div class="d-flex gap-2">
              <a href="{% url 'reject_dress' dress.id %}" class="btn btn-sm btn-danger flex-
grow-1">
                <svg width="14" height="14" class="me-1"><use
xlink:href="#close"></use></svg>
                Revoke
              </a>
            </div>
            {% else %}
            <div class="d-flex gap-2">
              <a href="{% url 'approve_dress' dress.id %}" class="btn btn-sm btn-success flex-
grow-1">
                <svg width="14" height="14" class="me-1"><use
xlink:href="#check"></use></svg>
                Approve
              </a>
            </div>
            {% endif %}
          </div>
        </div>
      </div>
```

```
        {% endfor %}
      </div>
      {% else %}
      <div class="text-center py-5">
        <svg width="64" height="64" class="text-muted mb-3"><use xlink:href="#shopping-
bag"></use></svg>
        <h4>No Dresses Found</h4>
        <p class="text-muted">The inventory is currently empty.</p>
      </div>
      {% endif %}
    </div>
</section>
{% endblock %}
```

4. DASHBOARD.HTML

```
{% extends 'base.html' %}
{% load static %}

{% block content %}
<!-- Hero Section -->
<section id="admin-hero" class="bg-light py-5">
    <div class="container">
      <div class="row justify-content-center">
        <div class="col-md-10 text-center" data-aos="fade-up">
          <h1 class="section-title text-uppercase mt-4">Admin Dashboard</h1>
          <p class="lead">Manage platform content, approve designs, and oversee operations</p>
        </div>
      </div>
    </div>
</section>

<!-- Stats Section -->
<section class="py-4 bg-white border-bottom">
    <div class="container">
      <div class="row g-3">
        <div class="col-md-3">
          <div class="card border-0 shadow-sm text-center h-100">
            <div class="card-body py-4">
              <svg width="32" height="32" class="text-warning mb-2"><use
xlink:href="#calendar"></use></svg>
```

```
                <h3 class="mb-0">{{ dresses|length }}</h3>
                <small class="text-muted text-uppercase">Pending Approvals</small>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm text-center h-100">
            <div class="card-body py-4">
                <svg width="32" height="32" class="text-success mb-2"><use
xlink:href="#check"></use></svg>
                <h3 class="mb-0">{{ approved_count }}</h3>
                <small class="text-muted text-uppercase">Approved This Week</small>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm text-center h-100">
            <div class="card-body py-4">
                <svg width="32" height="32" class="text-danger mb-2"><use
xlink:href="#close"></use></svg>
                <h3 class="mb-0">{{ rejected_count }}</h3>
                <small class="text-muted text-uppercase">Rejected</small>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm text-center h-100">
            <div class="card-body py-4">
                <svg width="32" height="32" class="text-primary mb-2"><use
xlink:href="#user"></use></svg>
                <h3 class="mb-0">{{ designers_count }}</h3>
                <small class="text-muted text-uppercase">Active Designers</small>
            </div>
        </div>
    </div>
  </div>
 </div>
</section>

<!-- Pending Approvals Section -->
<section class="py-5">
```

```
<div class="container">
  <!-- Search and Filter Bar -->
  <div class="card border-0 shadow-sm mb-4" data-aos="fade-up">
    <div class="card-body">
      <form method="GET" action="">
        <div class="row g-3 align-items-end">
          <div class="col-md-4">
            <label class="form-label small text-uppercase fw-bold">Search</label>
            <input type="text" name="search" class="form-control" placeholder="Search by
name..." value="{{ request.GET.search }}">
          </div>
          <div class="col-md-3">
            <label class="form-label small text-uppercase fw-bold">Category</label>
            <select name="category" class="form-select">
              <option value="">All Categories</option>
              <option value="Casual" {% if request.GET.category == 'Casual'
%}selected{% endif %}>Casual</option>
              <option value="Formal" {% if request.GET.category == 'Formal'
%}selected{% endif %}>Formal</option>
              <option value="Party" {% if request.GET.category == 'Party' %}selected{%
endif %}>Party</option>
              <option value="Traditional" {% if request.GET.category == 'Traditional'
%}selected{% endif %}>Traditional</option>
            </select>
          </div>
          <div class="col-md-3">
            <label class="form-label small text-uppercase fw-bold">Designer</label>
            <select name="designer" class="form-select">
              <option value="">All Designers</option>
              {% for designer in all_designers %}
              <option value="{{ designer.id }}" {% if request.GET.designer ==
designer.id|stringformat:"s" %}selected{% endif %}>{{ designer.user.username }}</option>
              {% endfor %}
            </select>
          </div>
          <div class="col-md-2">
            <button type="submit" class="btn btn-dark w-100 text-uppercase">
              <svg width="16" height="16" class="me-1"><use
xlink:href="#search"></use></svg>
              Filter
            </button>
```

```
                </div>
            </div>
        </form>
    </div>
</div>

<div class="d-flex justify-content-between align-items-center mb-4">
    <h4 class="text-uppercase mb-0">Pending Approvals</h4>
    <div class="d-flex gap-2 align-items-center">
        {% if request.GET.search or request.GET.category or request.GET.designer %}
        <a href="{% url 'admin_dashboard' %}" class="btn btn-sm btn-outline-
secondary">Clear Filters</a>
        {% endif %}
        <span class="badge bg-warning text-dark">{{ dresses|length }} waiting</span>
    </div>
</div>

{% if dresses %}
<div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 row-cols-xl-4 g-4">
    {% for dress in dresses %}
    <div class="col" data-aos="fade-up">
        <div class="card h-100 shadow-sm border-0 product-card">
            <div class="position-relative overflow-hidden image-zoom-effect">
                {% if dress.image %}
                <img src="{{ dress.image.url }}" class="card-img-top" alt="{{ dress.name }}"
style="height: 280px; object-fit: cover;">
                {% else %}
                <div class="bg-light d-flex align-items-center justify-content-center" style="height:
280px;">
                    <svg width="48" height="48" class="text-muted"><use xlink:href="#shopping-
bag"></use></svg>
                </div>
                {% endif %}

                <div class="position-absolute top-0 end-0 m-2">
                    <span class="badge bg-warning text-dark">Pending</span>
                </div>
            </div>

            <div class="card-body">
```

```
                <small class="text-muted text-uppercase d-block mb-1">{{ dress.category }}</small>
                <h6 class="card-title mb-2">{{ dress.name }}</h6>

                <div class="row g-2 mb-3">
                  <div class="col-6">
                    <small class="text-muted d-block">Price</small>
                    <strong class="text-primary">₹{{ dress.price }}</strong>
                  </div>
                  <div class="col-6">
                    <small class="text-muted d-block">Designer</small>
                    <span class="small">{{ dress.designer.user.username }}</span>
                  </div>
                </div>

                <div class="bg-light p-2 rounded small">
                  <div class="d-flex justify-content-between mb-1">
                    <span>Skin:</span>
                    <strong>{{ dress.suitable_skin_tone }}</strong>
                  </div>
                  <div class="d-flex justify-content-between">
                    <span>Body:</span>
                    <strong>{{ dress.suitable_body_type }}</strong>
                  </div>
                </div>
              </div>

              <div class="card-footer bg-white border-top p-3">
                <div class="d-flex gap-2">
                  <a href="{% url 'approve_dress' dress.id %}" class="btn btn-sm btn-success flex-grow-1">
                    <svg width="14" height="14" class="me-1"><use xlink:href="#check"></use></svg>
                    Approve
                  </a>
                  <a href="{% url 'reject_dress' dress.id %}" class="btn btn-sm btn-outline-danger flex-grow-1">
                    Reject
                  </a>
                </div>
              </div>
```

```
            </div>
          </div>
          {% endfor %}
        </div>
        {% else %}
        <div class="text-center py-5" data-aos="fade-up">
          <svg width="64" height="64" class="text-success mb-3"><use xlink:href="#check"></use></svg>
          <h4>All Caught Up!</h4>
          <p class="text-muted">No pending designs to review at this moment.</p>
        </div>
        {% endif %}
    </div>
</section>
{% endblock %}MODELS.PY
from django.db import models
from django.contrib.auth.models import AbstractUser

class CustomUser(AbstractUser):
    ROLE_CHOICES = (
        ('admin', 'Admin'),
        ('user', 'User'),
    )
    STATUS_CHOICES = (
        ('pending', 'Pending'),
        ('approved', 'Approved'),
        ('blocked', 'Blocked'),
    )
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='user')
    approval_status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='pending')
    phone_no = models.CharField(max_length=15, blank=True, null=True)
    visible_password = models.CharField(max_length=255, blank=True, null=True)

    def __str__(self):
        return self.username

class AuditLog(models.Model):
    user = models.ForeignKey(CustomUser, on_delete=models.CASCADE, null=True, blank=True, related_name='audit_logs')
```

```
    admin_user = models.ForeignKey(CustomUser, on_delete=models.SET_NULL, null=True, related_name='admin_actions')
    document = models.ForeignKey('Document', on_delete=models.SET_NULL, null=True, blank=True, related_name='audit_logs')
    action = models.CharField(max_length=255)
    action_type = models.CharField(max_length=100, null=True, blank=True) # Upload, Fingerprint, Integrity Check, etc.
    document_name = models.CharField(max_length=255, null=True, blank=True)
    status = models.CharField(max_length=50, null=True, blank=True) # Success, Tampered, Verified, etc.
    similarity_score = models.FloatField(null=True, blank=True)
    target_user = models.CharField(max_length=255, null=True, blank=True) # Kept for backward compatibility
    timestamp = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user.username if self.user else 'System'} - {self.action} - {self.timestamp}"

class Document(models.Model):
    user = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='documents')
    title = models.CharField(max_length=255)
    file = models.FileField(upload_to='documents/')
    file_type = models.CharField(max_length=50) # To store extension like .pdf, .docx
    file_size = models.BigIntegerField() # Storing size in bytes for limits
    sha256_hash = models.CharField(max_length=64)
    cognitive_fingerprint = models.TextField(null=True, blank=True) # Global fingerprint
    section_fingerprints = models.TextField(null=True, blank=True) # JSON list of section-level fingerprints
    tamper_report = models.TextField(null=True, blank=True) # JSON details of last verification result
    status = models.CharField(max_length=50, default='Secure') # Secure, Processed, Tampered, Verified, etc
    uploaded_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title

class SecurityAlert(models.Model):
    document = models.ForeignKey(Document, on_delete=models.CASCADE, related_name='security_alerts')
```

```
    user = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='notifications')
    similarity_score = models.FloatField()
    is_read = models.BooleanField(default=False)
    timestamp = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Tamper Alert: {self.document.title} ({self.similarity_score}%)"

class DocumentPermission(models.Model):
    document = models.ForeignKey(Document, on_delete=models.CASCADE, related_name='permissions')
    user = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='document_permissions')
    can_view = models.BooleanField(default=False)
    can_download = models.BooleanField(default=False)
    can_edit = models.BooleanField(default=False)
    can_share = models.BooleanField(default=False)
    granted_by = models.ForeignKey(CustomUser, on_delete=models.SET_NULL, null=True, related_name='granted_permissions')
    granted_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        unique_together = ('document', 'user')

    def __str__(self):
        return f"{self.user.username} -> {self.document.title}"

class RetrievalRequest(models.Model):
    STATUS_CHOICES = (
        ('Pending', 'Pending'),
        ('Approved', 'Approved'),
        ('Rejected', 'Rejected'),
    )
    document = models.ForeignKey(Document, on_delete=models.CASCADE, related_name='retrieval_requests')
    requested_by = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='retrieval_requests')
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='Pending')
    requested_at = models.DateTimeField(auto_now_add=True)
```

```
    reviewed_by = models.ForeignKey(CustomUser, on_delete=models.SET_NULL, null=True, blank=True, related_name='reviewed_requests')
    reviewed_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ['-requested_at']

    def __str__(self):
        return f"{self.requested_by.username} -> {self.document.title} [{self.status}]"

class ShareRequest(models.Model):
    STATUS_CHOICES = (
        ('Pending', 'Pending'),
        ('Approved', 'Approved'),
        ('Rejected', 'Rejected'),
    )
    document = models.ForeignKey(Document, on_delete=models.CASCADE, related_name='share_requests')
    requested_by = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='share_requests_made')
    target_user = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='share_requests_received')
    can_view = models.BooleanField(default=False)
    can_download = models.BooleanField(default=False)
    can_edit = models.BooleanField(default=False)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='Pending')
    requested_at = models.DateTimeField(auto_now_add=True)
    reviewed_by = models.ForeignKey(CustomUser, on_delete=models.SET_NULL, null=True, blank=True, related_name='reviewed_share_requests')
    reviewed_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ['-requested_at']

    def __str__(self):
        return f"{self.requested_by.username} shares {self.document.title} with {self.target_user.username} [{self.status}]"
```

9. BIBLIOGRAPHY

BIBLIOGRAPHY

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. A comprehensive resource on machine learning and deep learning techniques used for intelligent data analysis and pattern recognition.
- Jurafsky, D., & Martin, J. H. (2021). Speech and Language Processing. Pearson. Provides detailed concepts of Natural Language Processing used for analysing semantic and contextual features of documents.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.
  Discusses methods for extracting and analysing textual information, relevant for document feature extraction.
- Rivest, R. (1992). The MD5 Message-Digest Algorithm. RFC 1321. Introduces hashing techniques for data integrity verification, relevant to document security mechanisms.
- Stallings, W. (2017). Cryptography and Network Security: Principles and Practice. Pearson. Provides concepts of encryption and data security used in protecting digital documents.
- OpenCV Documentation. (n.d.). Image and Document Processing. Retrieved from https://docs.opencv.org
  A widely used library for processing images and extracting features from visual documents.
- Flask Documentation. (n.d.). Flask Web Framework. Retrieved from https://flask.palletsprojects.com
  Backend framework used for developing the web-based interface of the system.
- spaCy Documentation. (n.d.). Natural Language Processing Library. Retrieved from https://spacy.io
  Used for extracting semantic and contextual features from document content.
- MySQL Documentation. (n.d.). Relational Database Management System. Retrieved from https://www.mysql.com
  Used for storing system data and managing document-related information securely.